

CSE 256, Winter 2026

Lecture 8: Machine Translation + Seq2Seq

Ndapa Nakashole, UCSD
29 January 2026





Recap: Language Modeling

A language model learns what sequences of words are *likely*

- ▶ Given what has already been seen
- ▶ e.g., Human infants drink _____

The Language modeling problem

Language modeling is the problem of assigning probabilities to sequences, $P(w_1, \dots, w_T)$, by repeatedly predicting what comes *next*:

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

- ▶ Everything that follows is a design choice about how to represent “what has already happened”

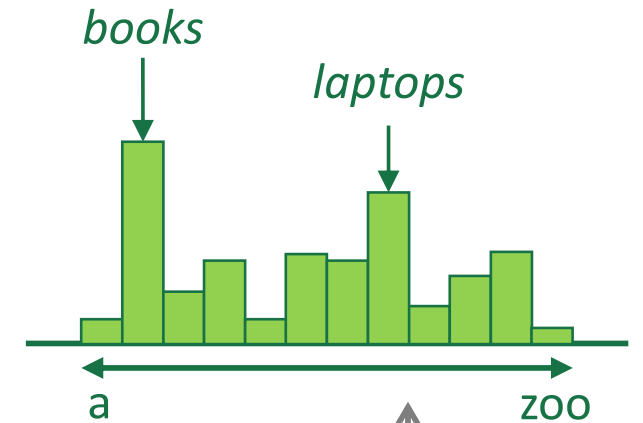


Recap: RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax} \left(U h^{(t)} + b_2 \right) \in \mathbb{R}^{|V|}$$

$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$



hidden states

$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right)$$

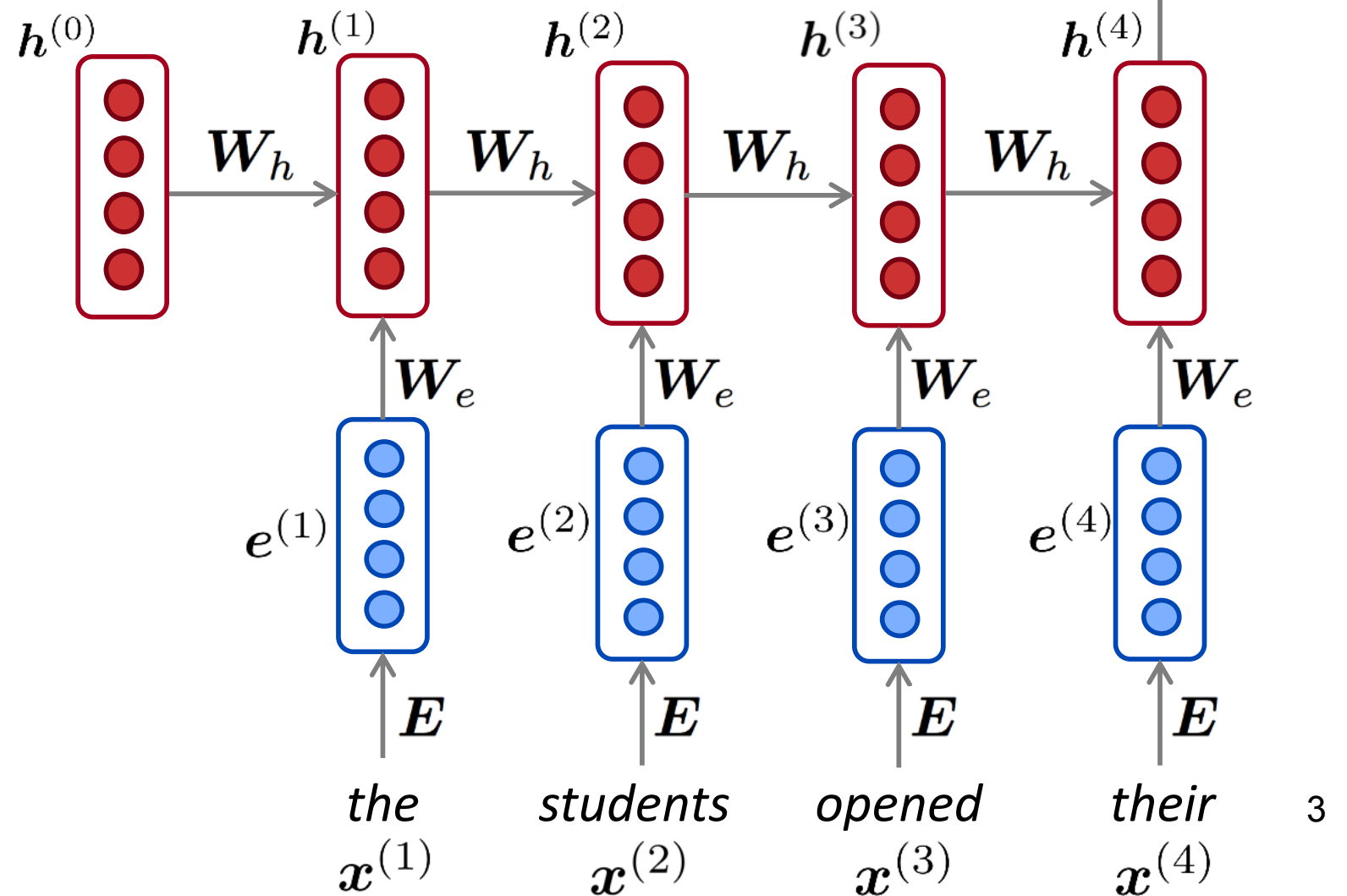
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$





Today

- New NLP task: **Machine Translation (MT)**
- MT new neural Network Architecture: **Seq2Seq** that builds on RNNs
- Seq2Seq motivates a new learning technique: the **Attention Mechanism**
- **Beam Search Decoding**



Machine Translation

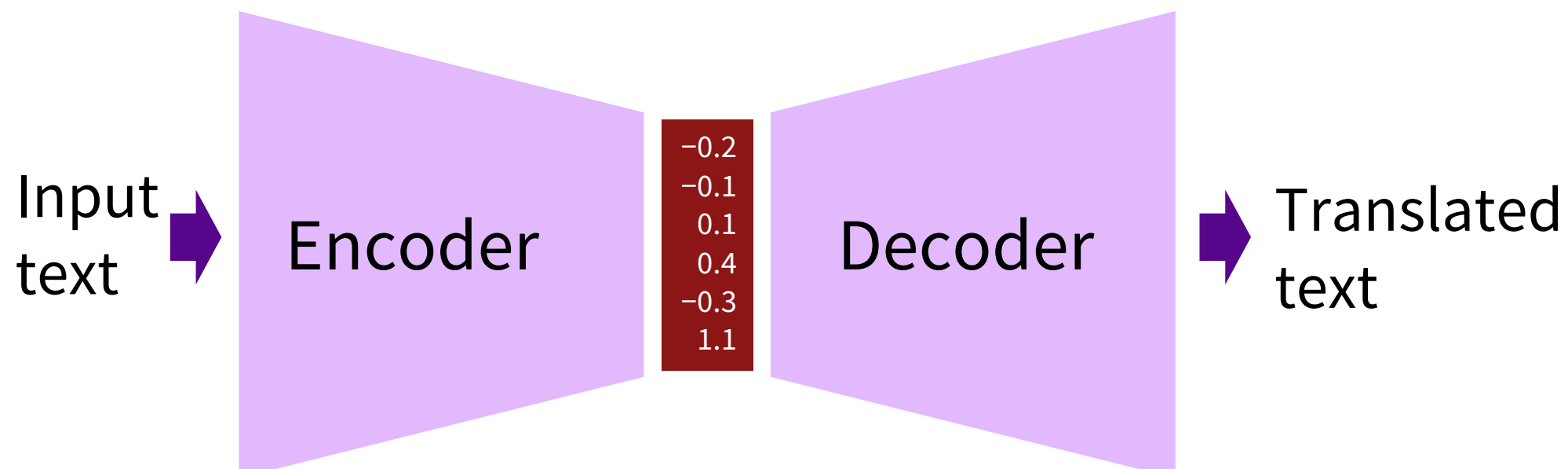
- MT is a classic test of language understanding
- It requires both language **analysis** and **generation**
- Large social/government/military needs
- Big in Europe and growing in other parts of the world





Neural MT

- **Neural Machine Translation (NMT)** is a way to do Machine Translation with a *single end-to-end neural network*
- The neural network architecture is called a **sequence-to-sequence** model (aka **seq2seq**) and it involves **two RNNs**





Neural Machine Translation (NMT)

The **sequence-to-sequence** model is an example of a **Conditional Language Model**

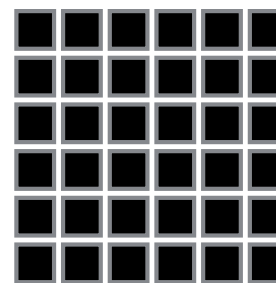
- **Language Model** because the decoder is predicting the next word of the target sentence y
- **Conditional** because its predictions are *also* conditioned on the source sentence x

Autoregressive Conditioned Prediction

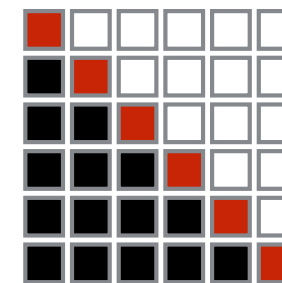
$$P(Y|X) = \prod_{i=1}^{|Y|} P(y_i|X, y_1, \dots, y_{i-1})$$

(e.g. seq2seq model)

Source X



Target Y



Question: How to **train** a NMT system?

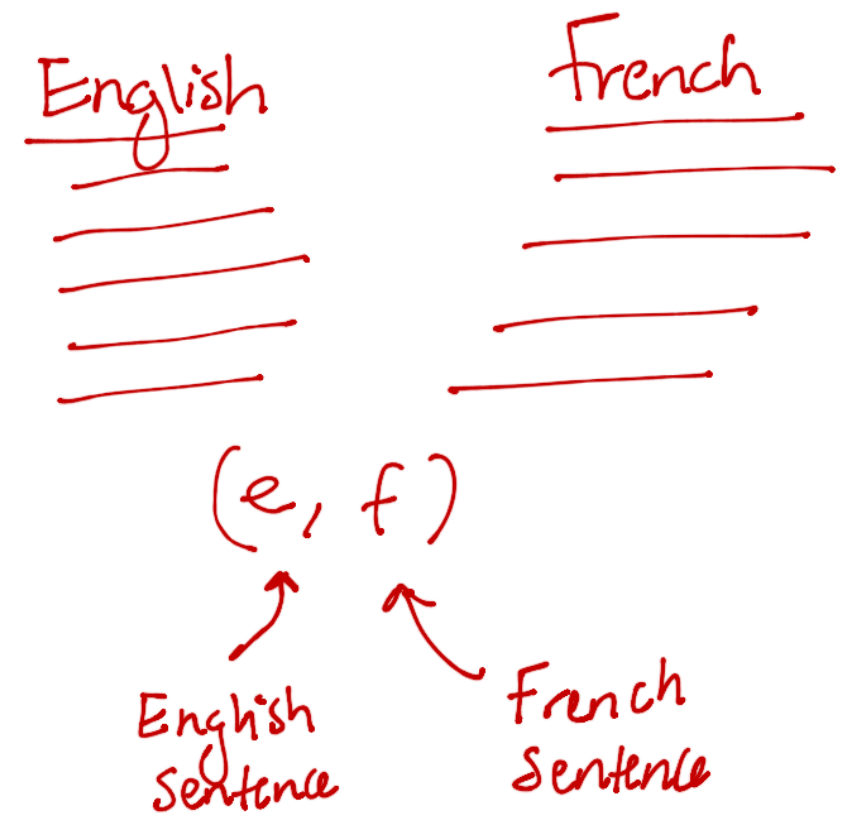
Answer: Get a big parallel corpus...



Machine Translation data

- Basic idea: Parallel corpora are available in several language pairs
 - treat translation as a supervised learning problem.

- Where does the data come from?
 - Canadian Parliament Proceedings
 - European Parliament
 - Bilingual Newspapers
 - Book translations
 - Movie Subtitles
 - TED talks





The Sequence-to-sequence model

German: Representieren wir diesen Satz.

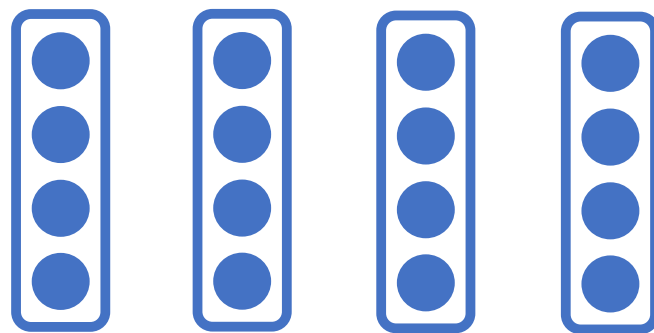
English: Let's represent this sentence.



The Sequence-to-sequence model

- Look up the embedding of each source word

Source language
word embeddings

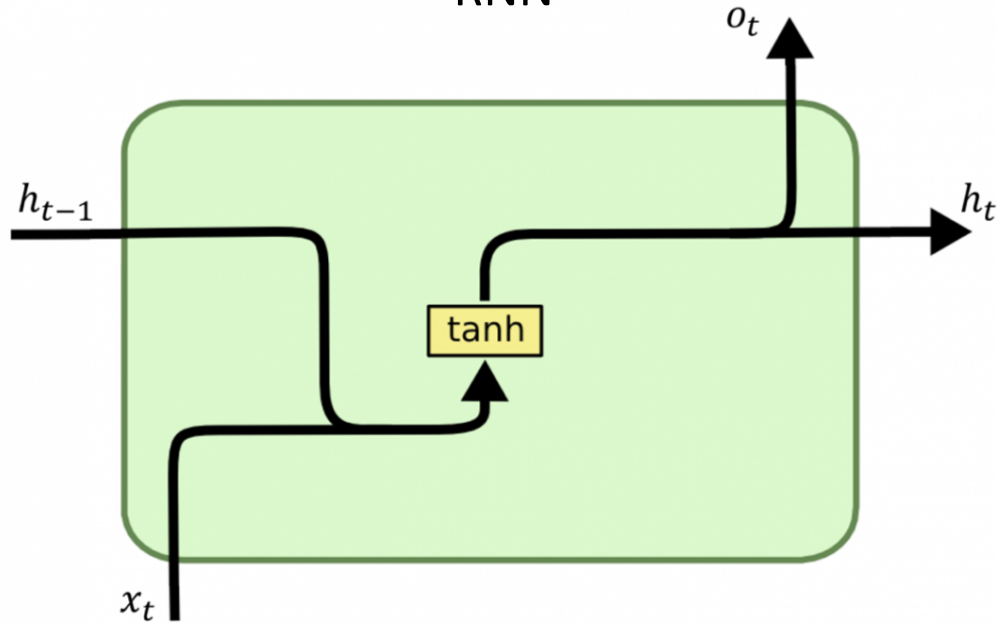


Let's represent this sentence

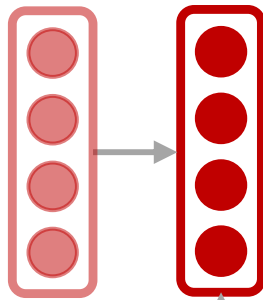


Seq2Seq: first hidden state

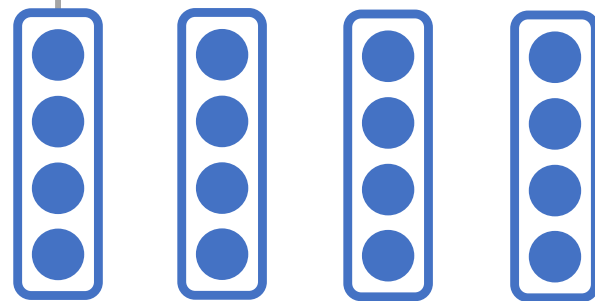
RNN



Encoder hidden
vectors

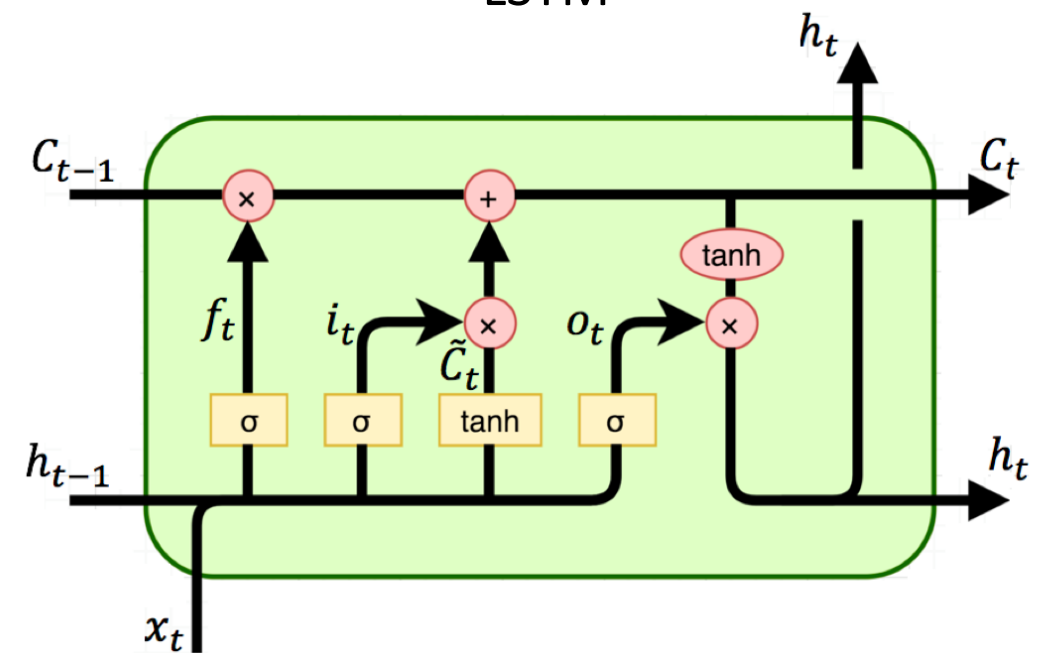


Source language
word embeddings



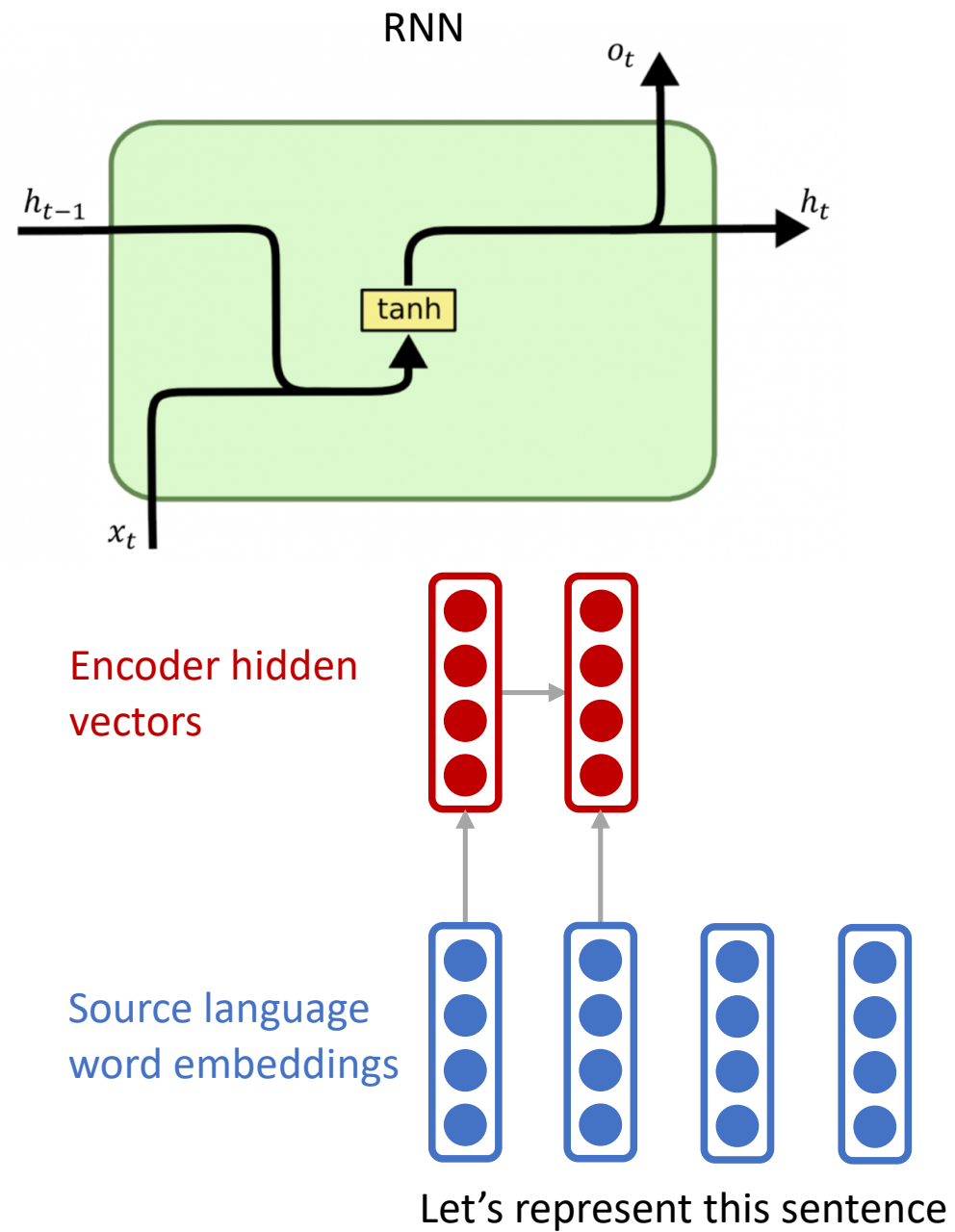
Let's represent this sentence

LSTM



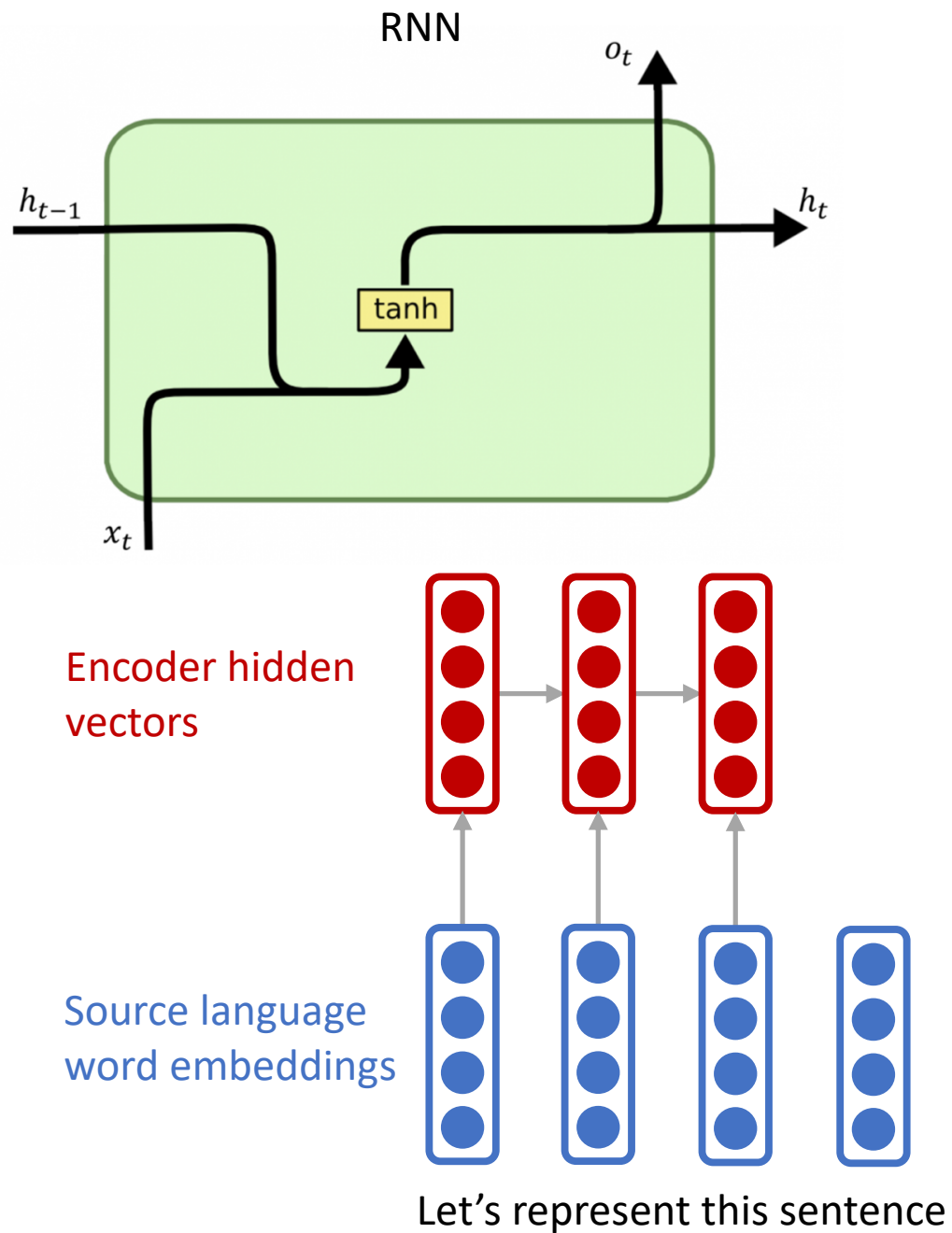


Seq2Seq: second encoder hidden state



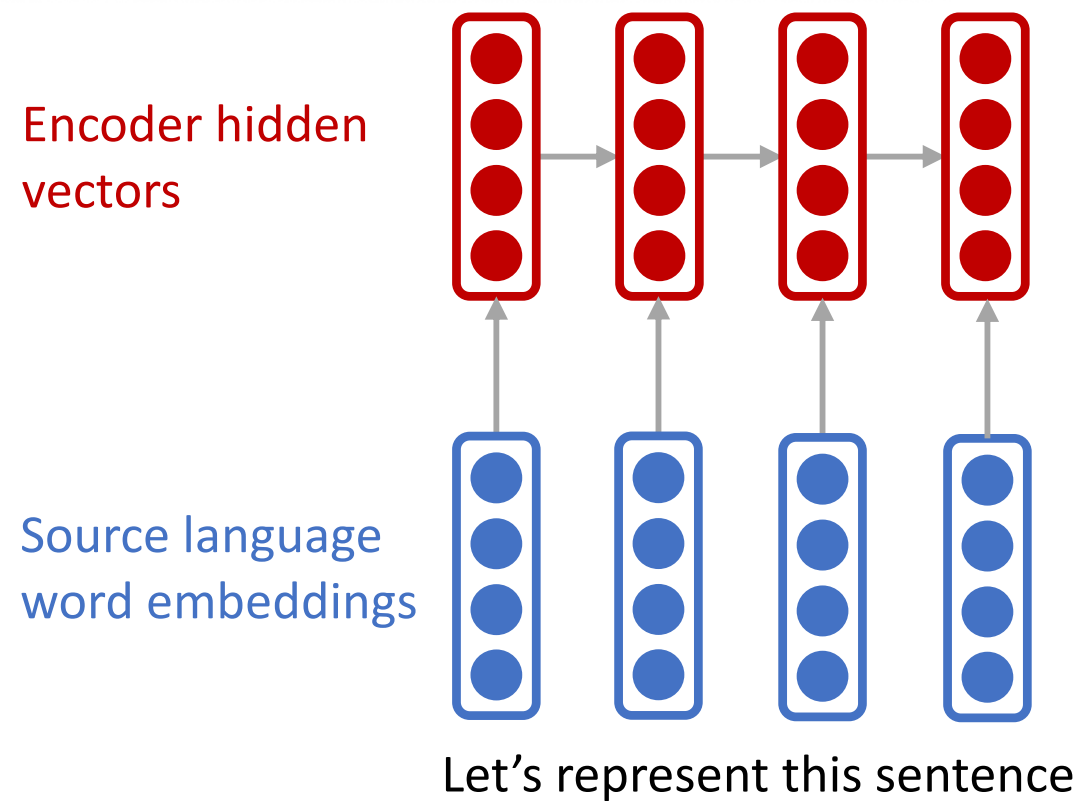
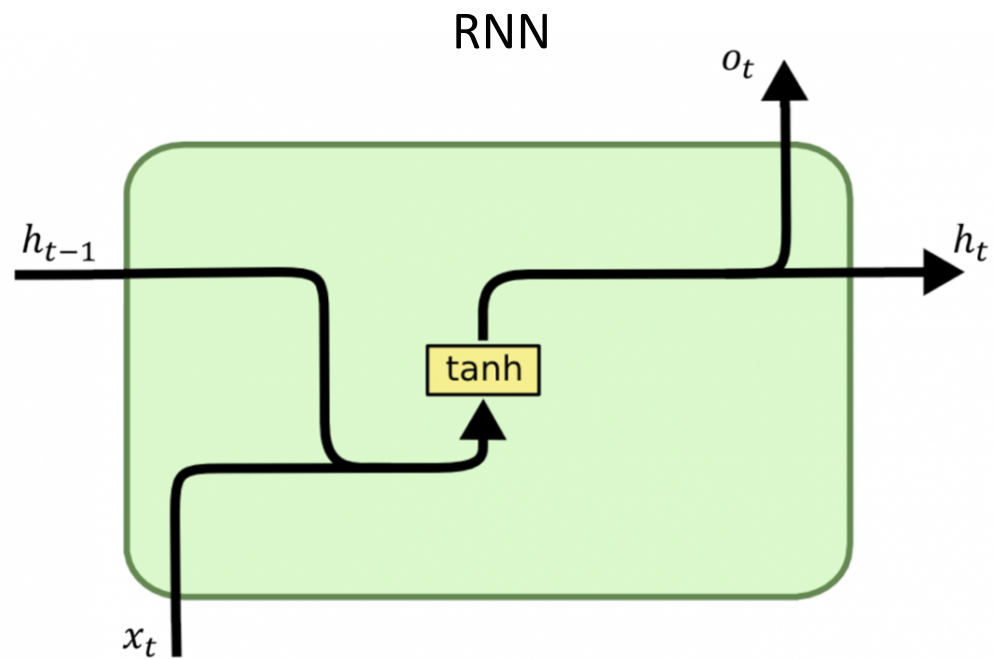


Seq2Seq: third encoder hidden state



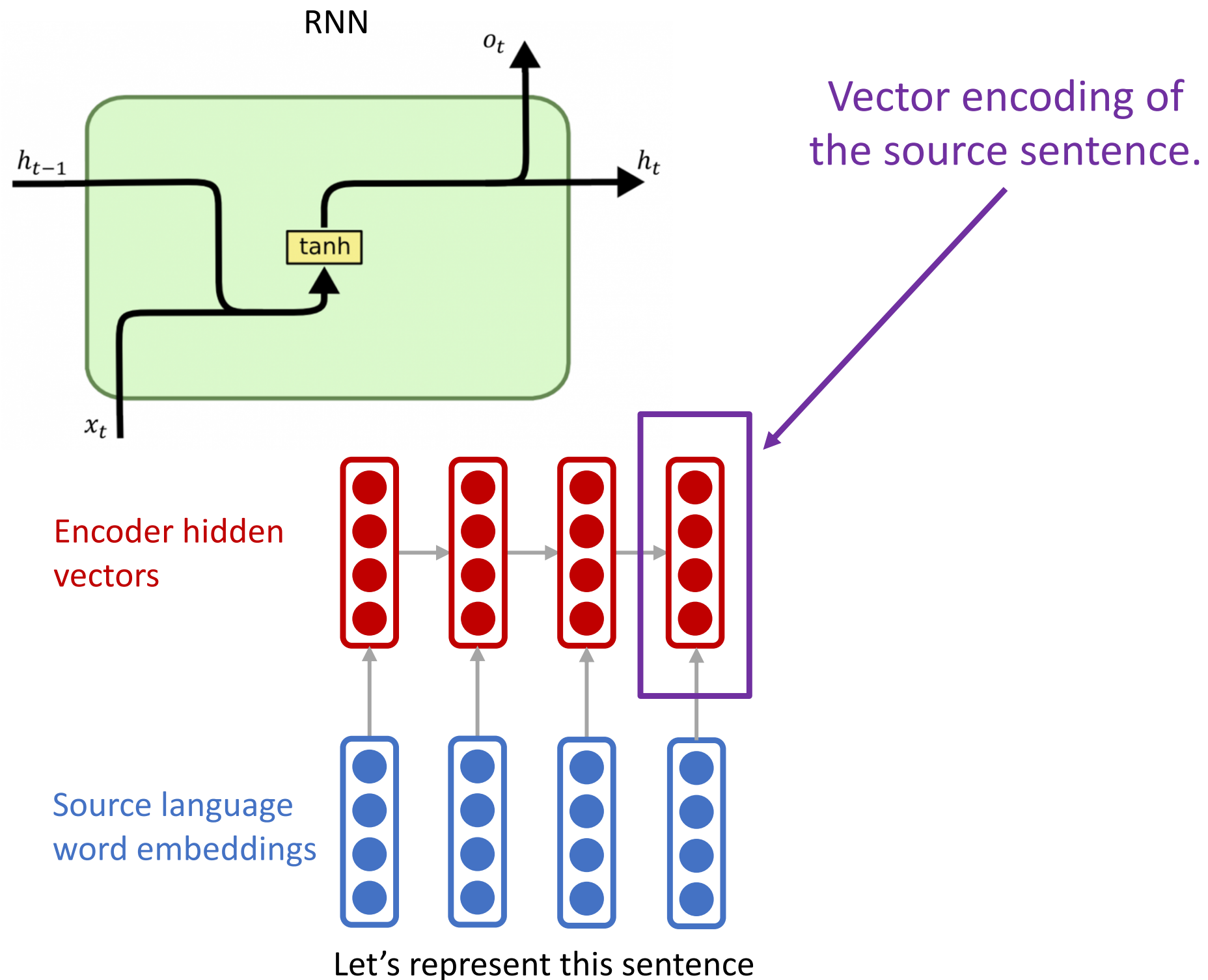


Seq2Seq: encoder hidden states



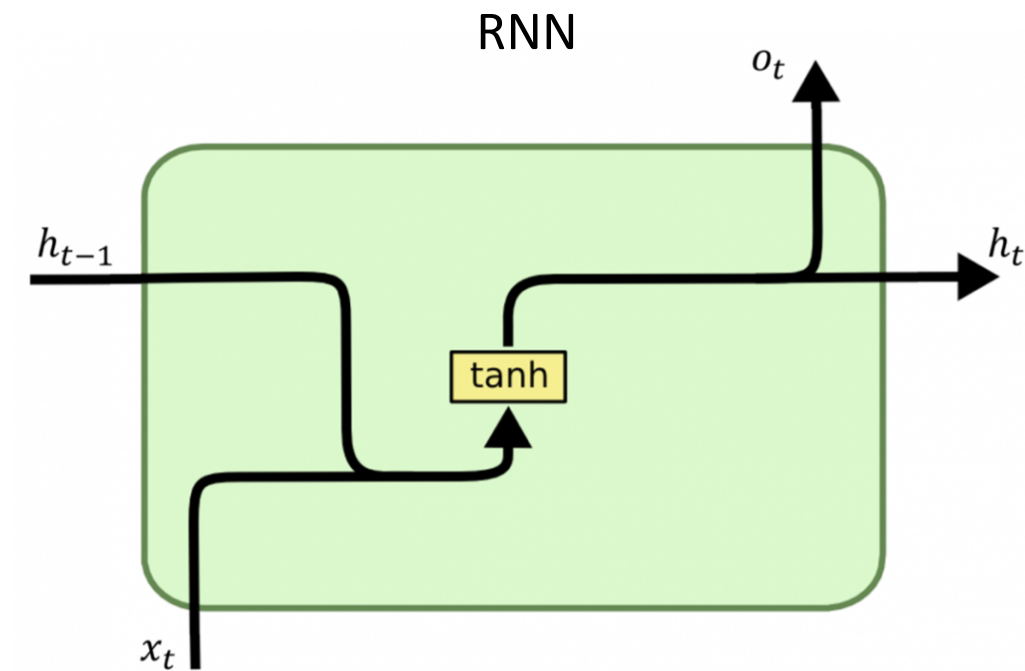


Seq2Seq: last hidden summarizes the source sentence

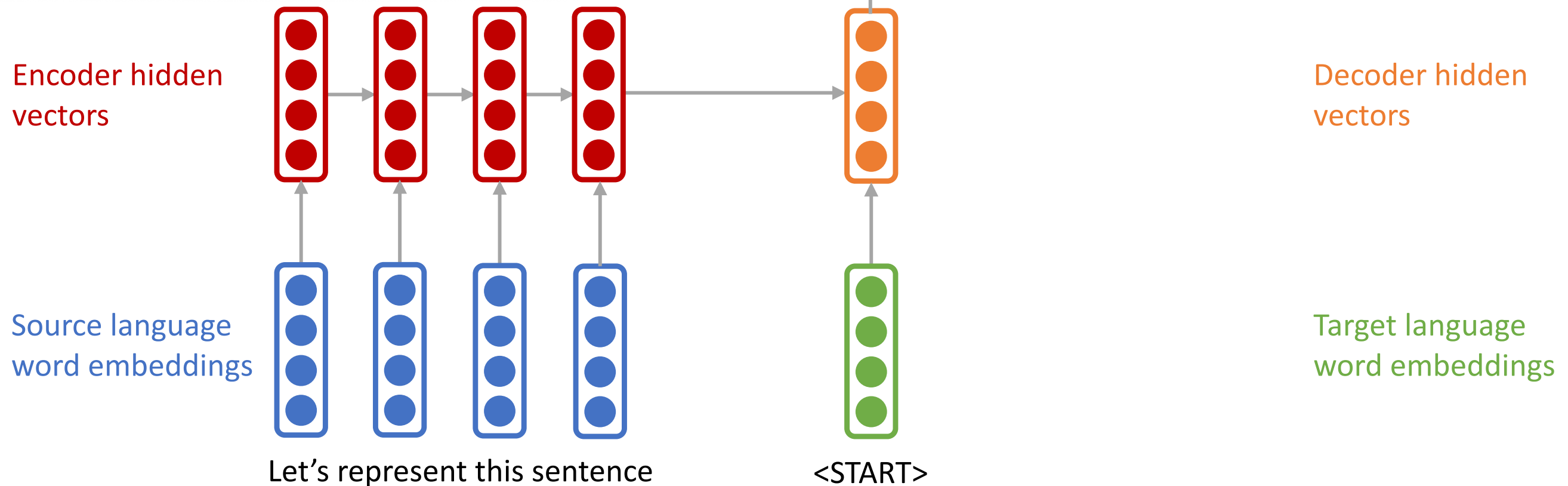




Seq2Seq: Decoder RNN

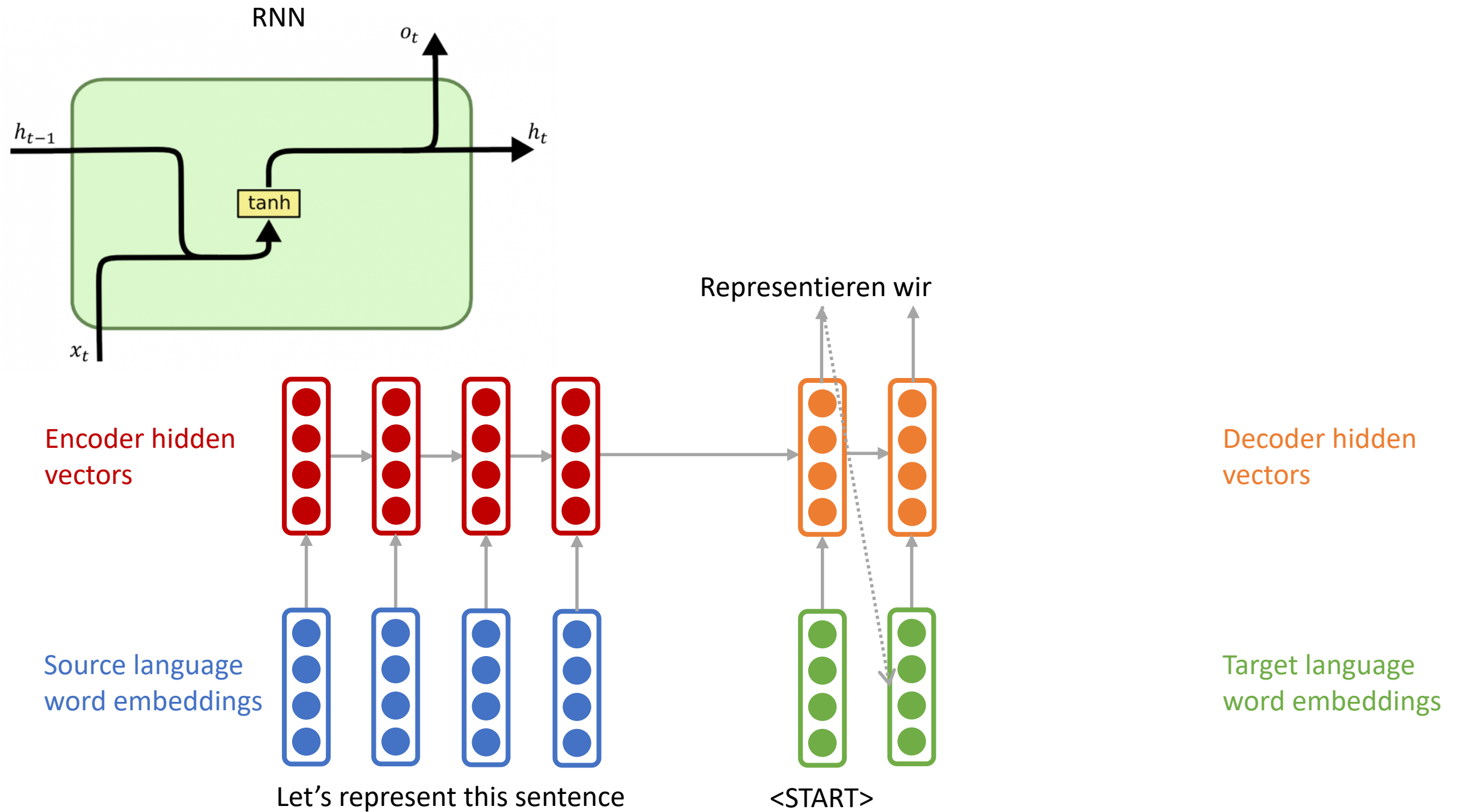


Output function reads the state vector at each time step, and generates a word (actually generates a distribution over words)





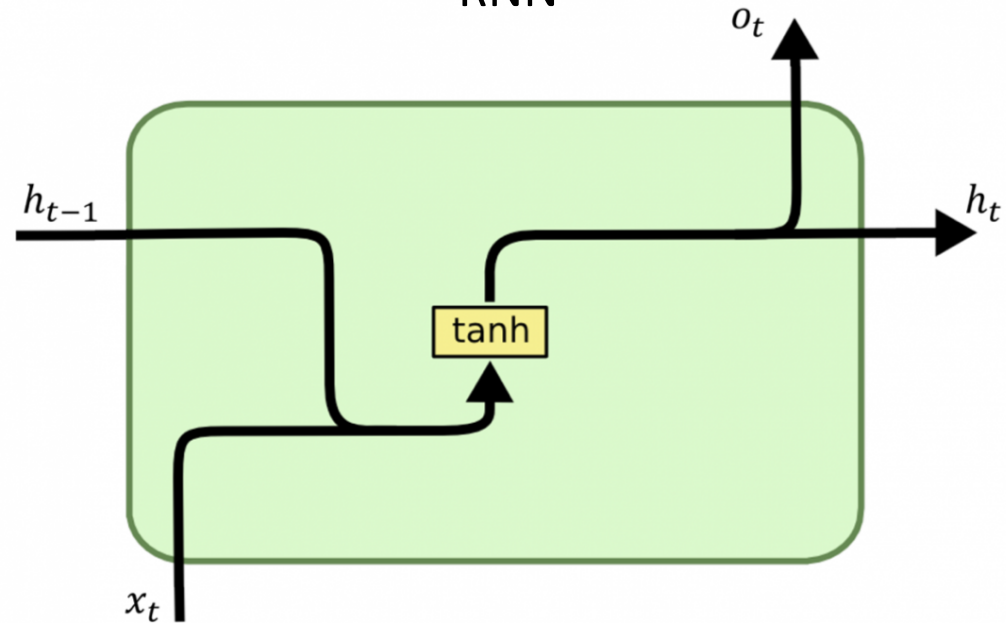
Seq2Seq: Decoder RNN





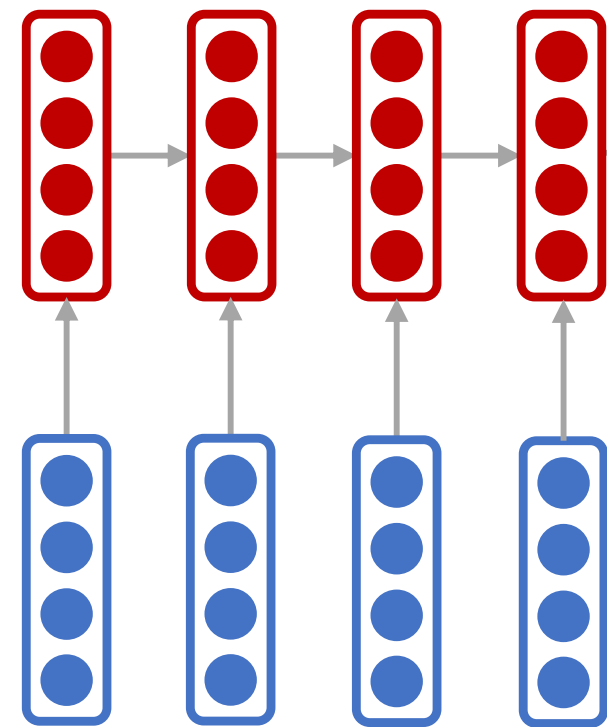
Seq2Seq: Decoder RNN

RNN



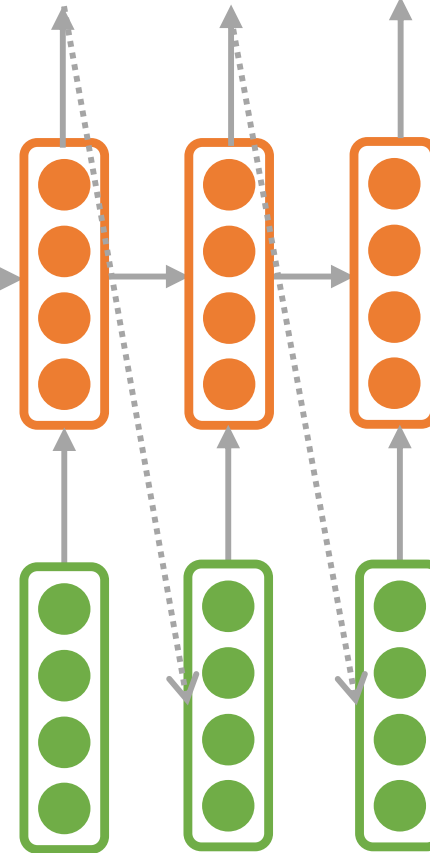
Encoder hidden
vectors

Source language
word embeddings



Let's represent this sentence

Representieren wir diesen



<START>

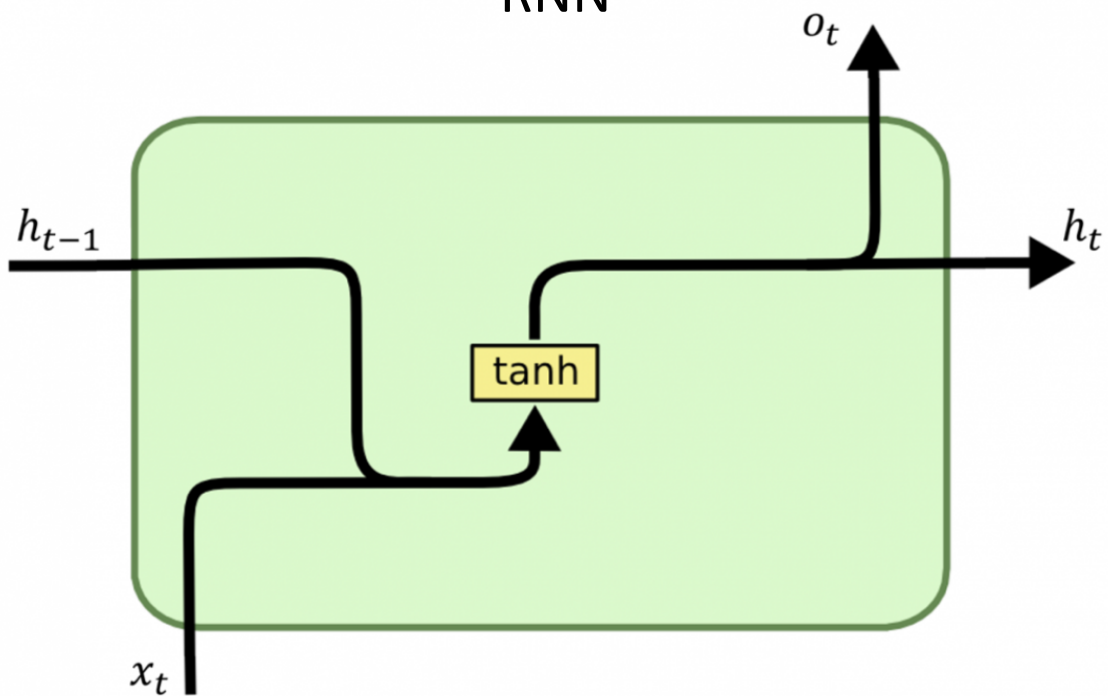
Decoder hidden
vectors

Target language
word embeddings



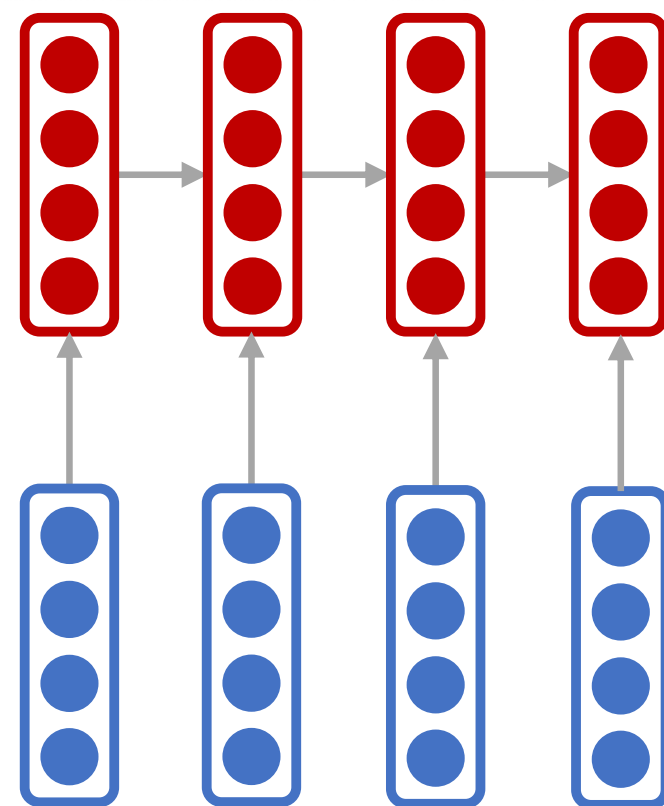
Seq2Seq: Decoder RNN

RNN



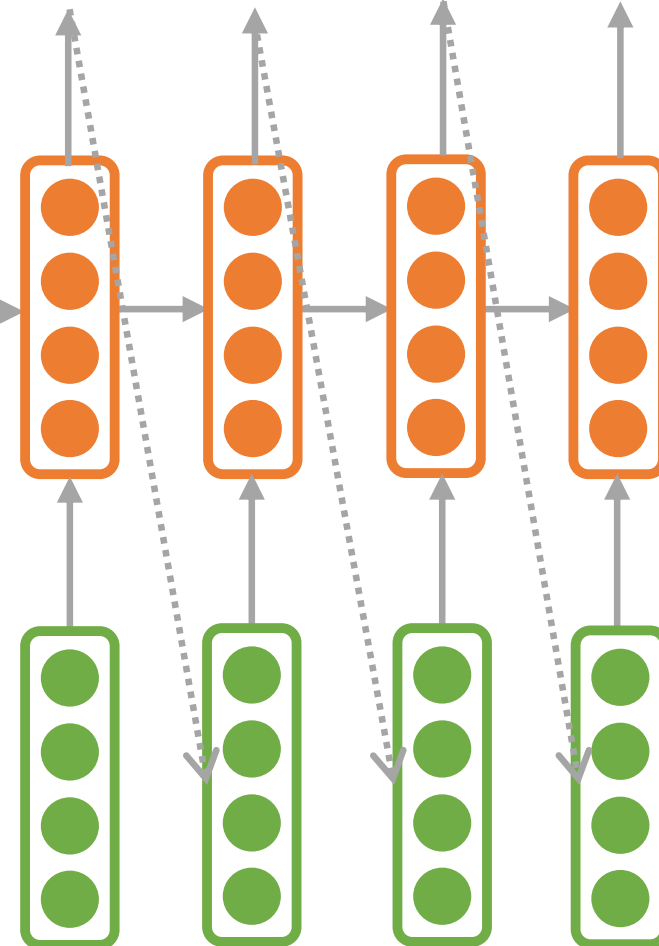
Encoder hidden
vectors

Source language
word embeddings



Let's represent this sentence

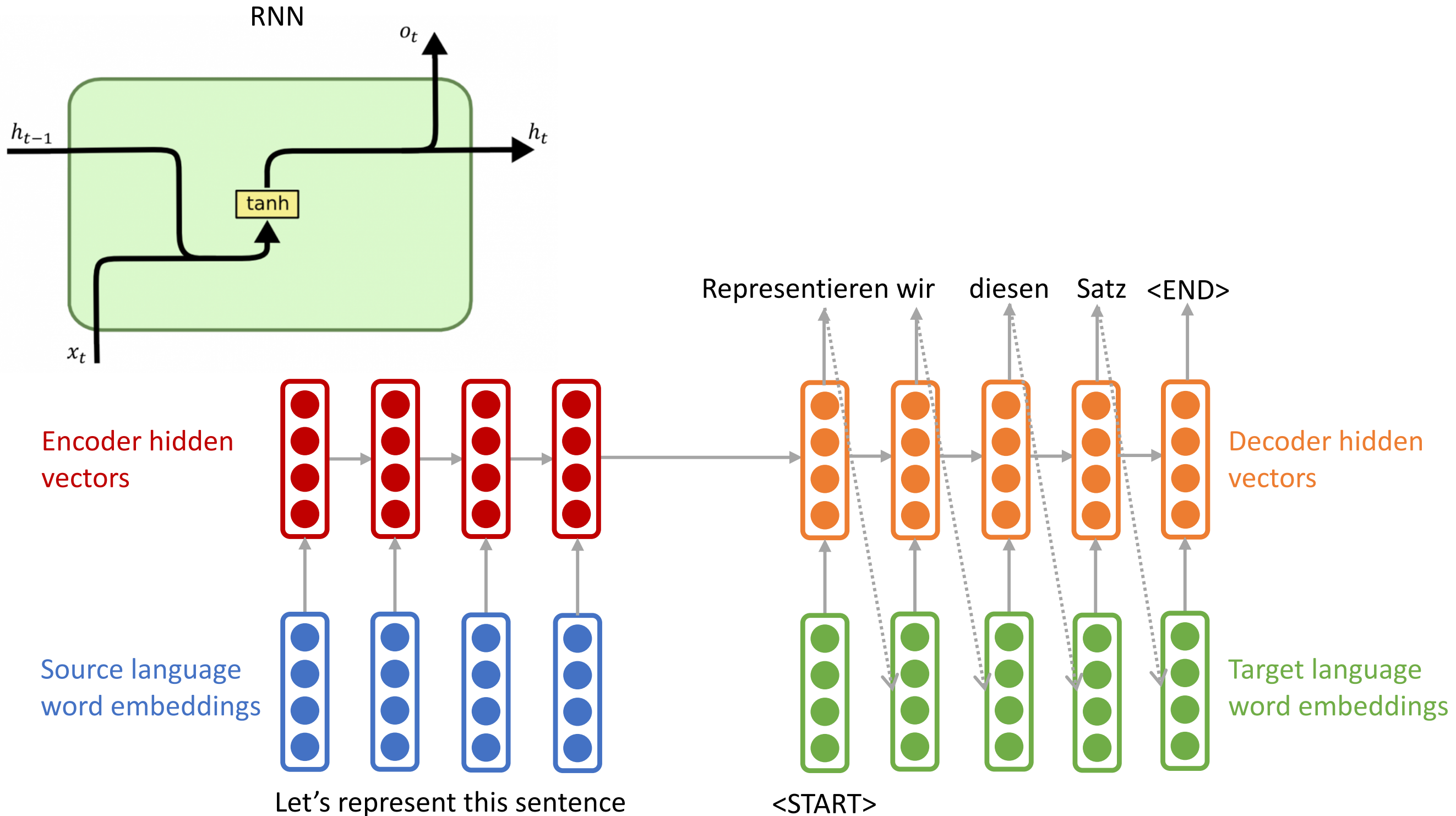
Representieren wir diesen Satz



<START>



Seq2Seq: Decoder RNN - emits END token





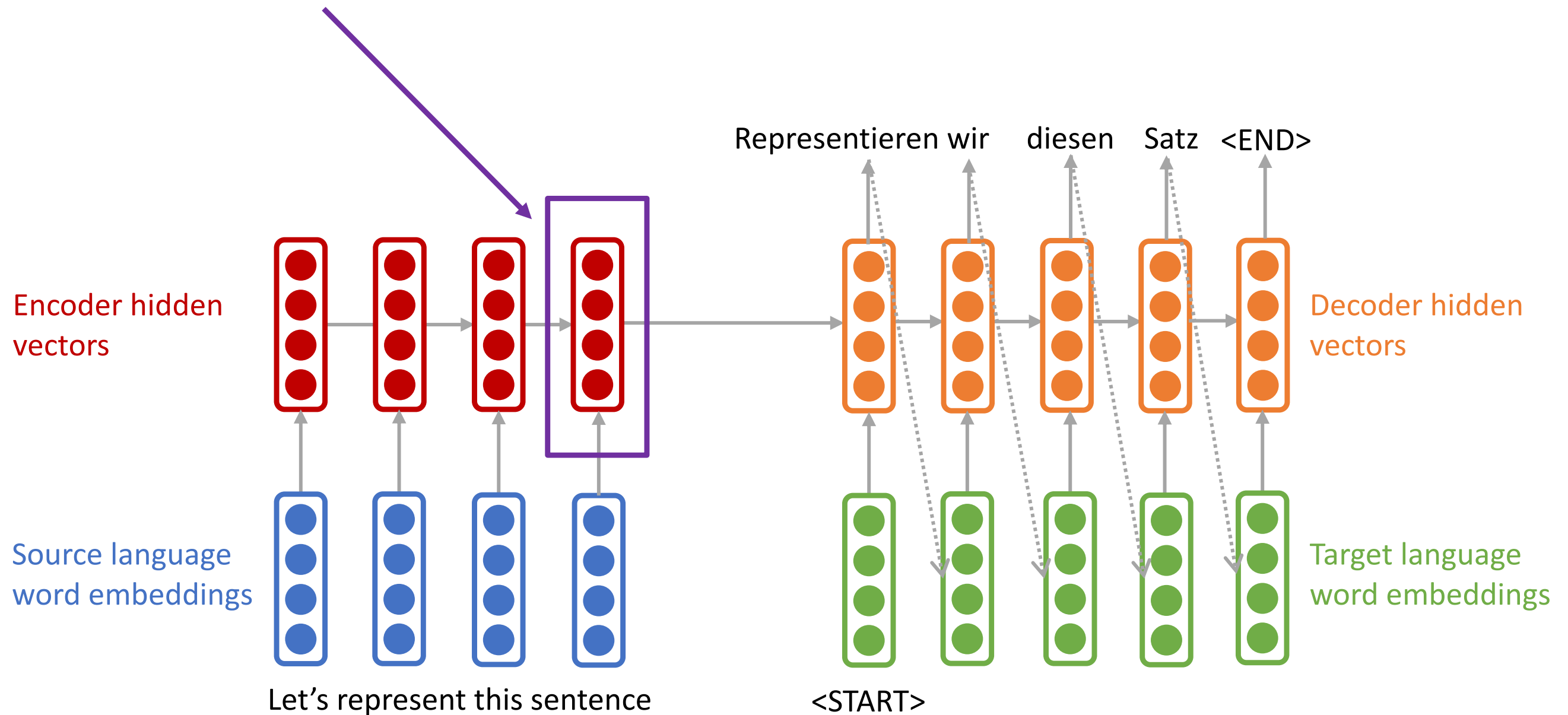
Vanilla Sequence-to-Sequence

- One look at the summary of the source sentence for the duration of translation



Seq2Seq: one summary of the source sentence

Vector encoding of
the source sentence.

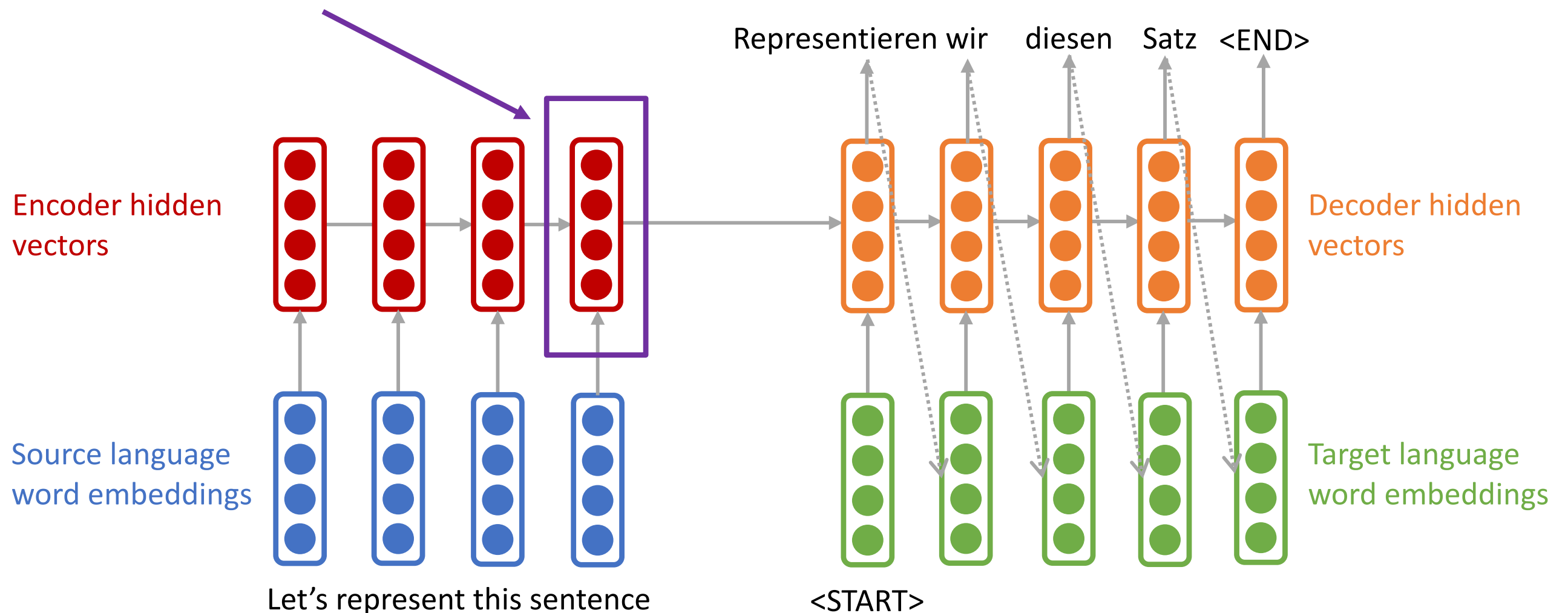




Seq2Seq: one summary == information bottleneck

Vector encoding of
the source sentence.

**Information
bottleneck!**





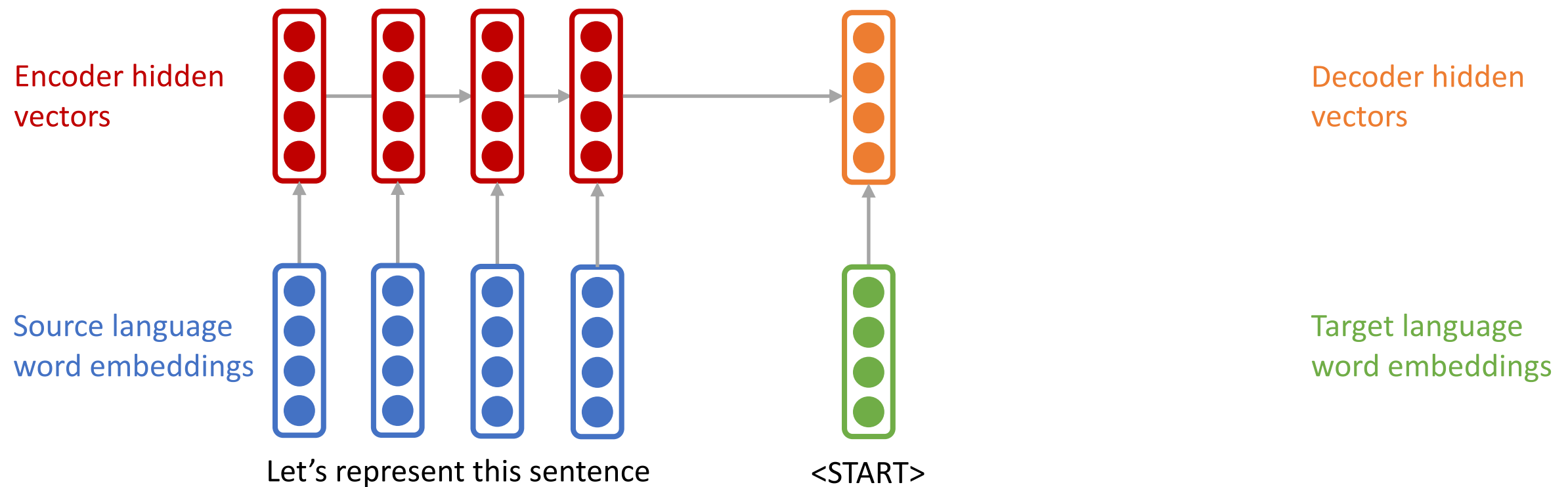
Sequence-to-Sequence with Attention

- May want to **look back at the source sentence from time to time** as we generate the translation
- Attention will allow us to **align** each output word with relevant input words



Seq2Seq with Attention

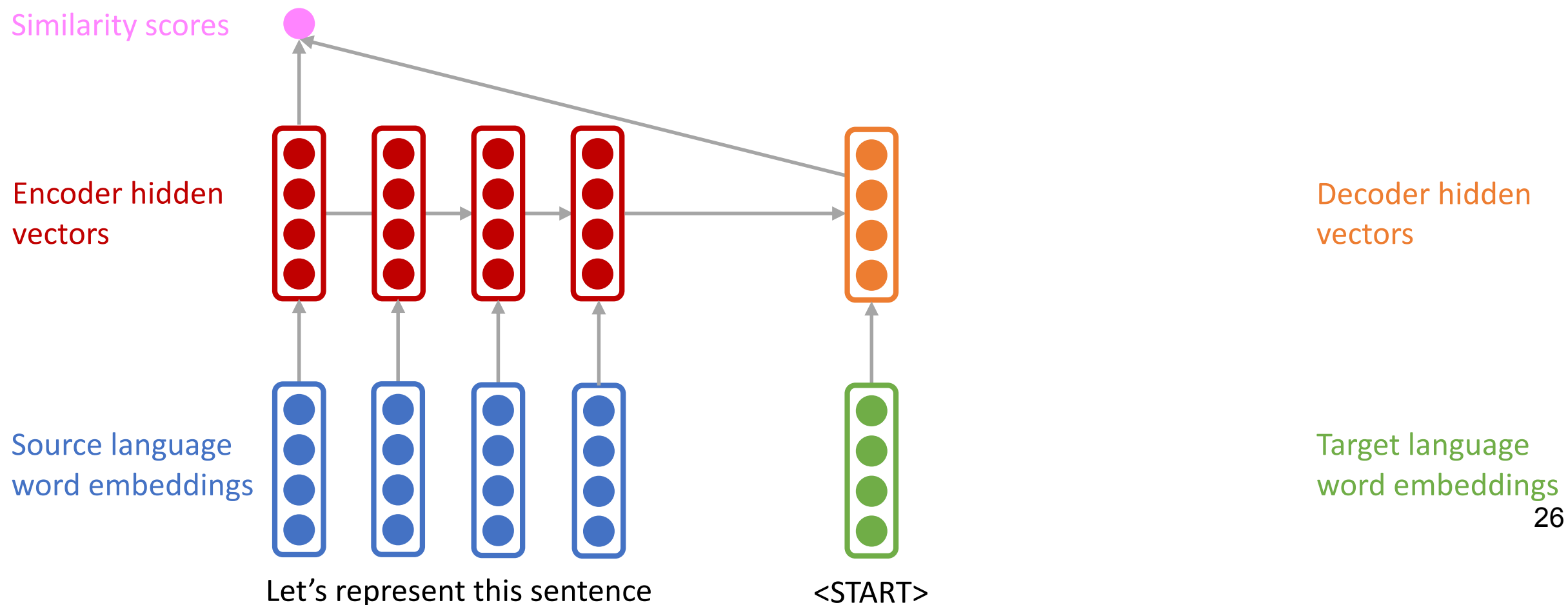
Generate 1st decoder
hidden vector as before.





Seq2Seq with Attention: similarity scores

Calculate a **similarity score** between the decoder hidden vector and all the encoder hidden vectors.





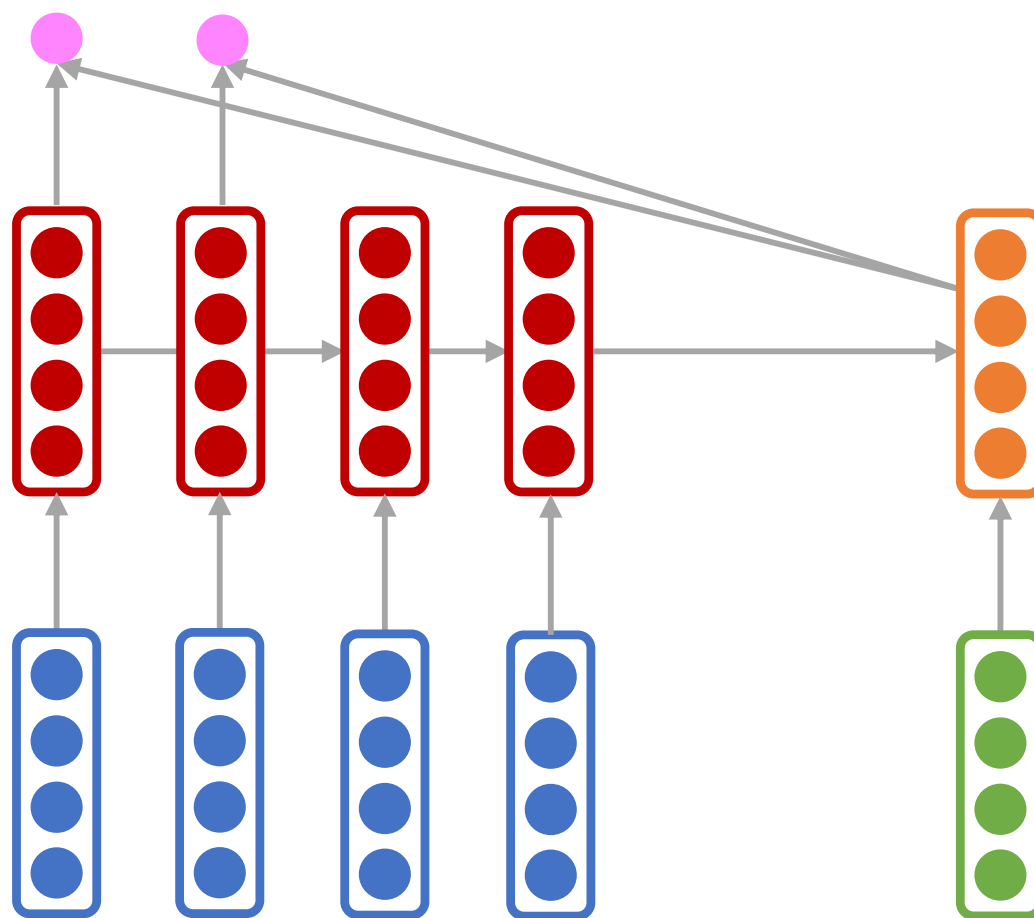
Seq2Seq with Attention: similarity scores

Calculate a **similarity score** between the decoder hidden vector and all the encoder hidden vectors.

Similarity scores

Encoder hidden vectors

Source language word embeddings



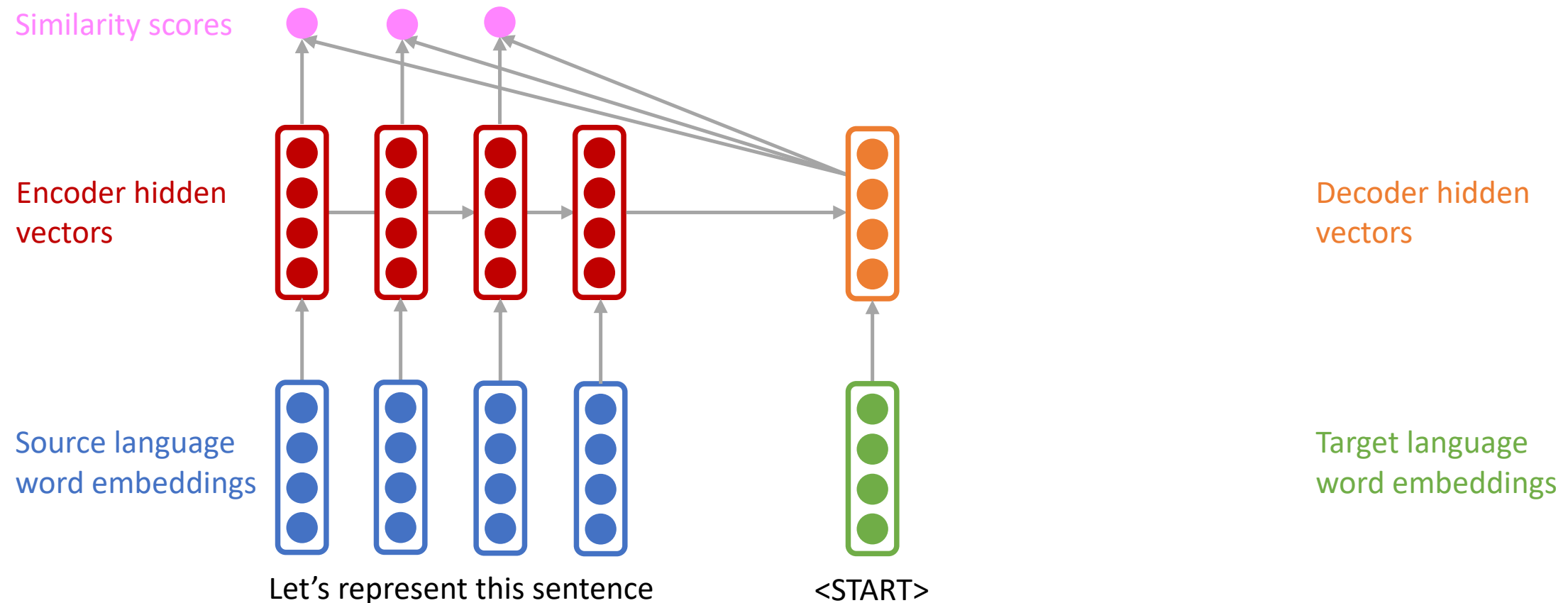
Decoder hidden vectors

Target language word embeddings



Seq2Seq with Attention: similarity scores

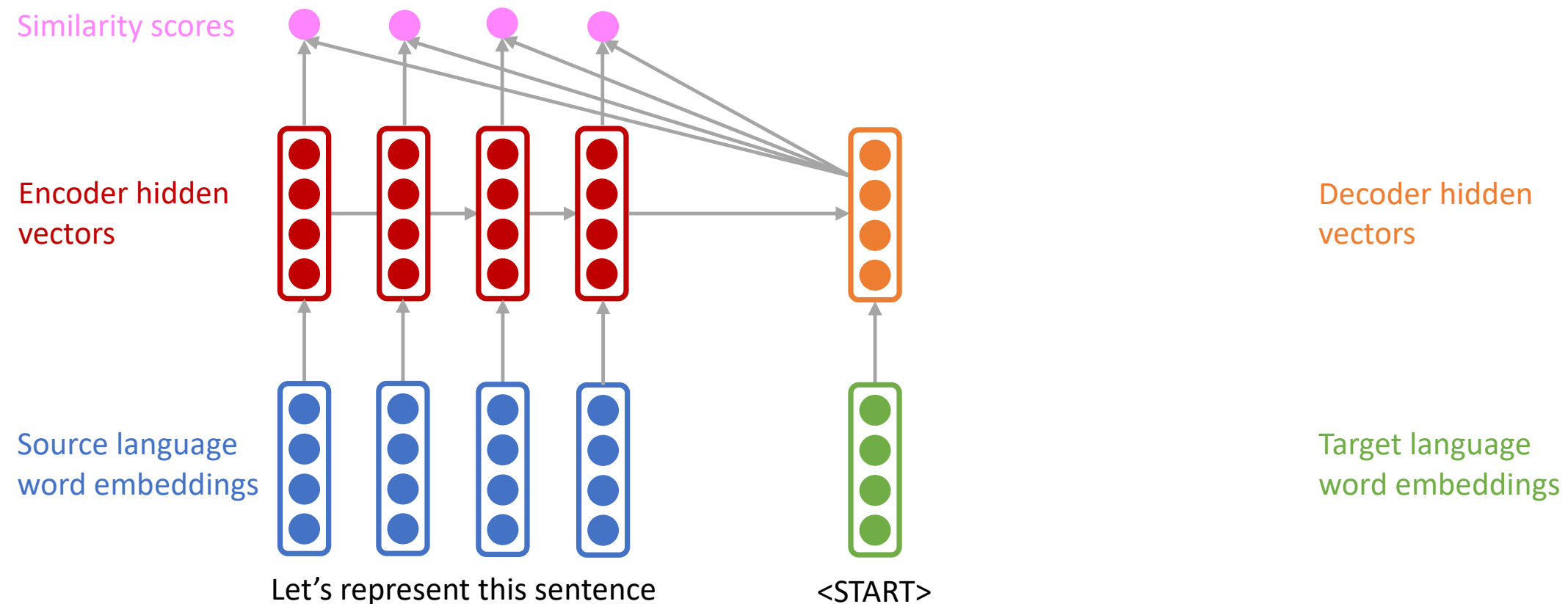
Calculate a **similarity score** between the decoder hidden vector and all the encoder hidden vectors.





Seq2Seq with Attention: similarity scores

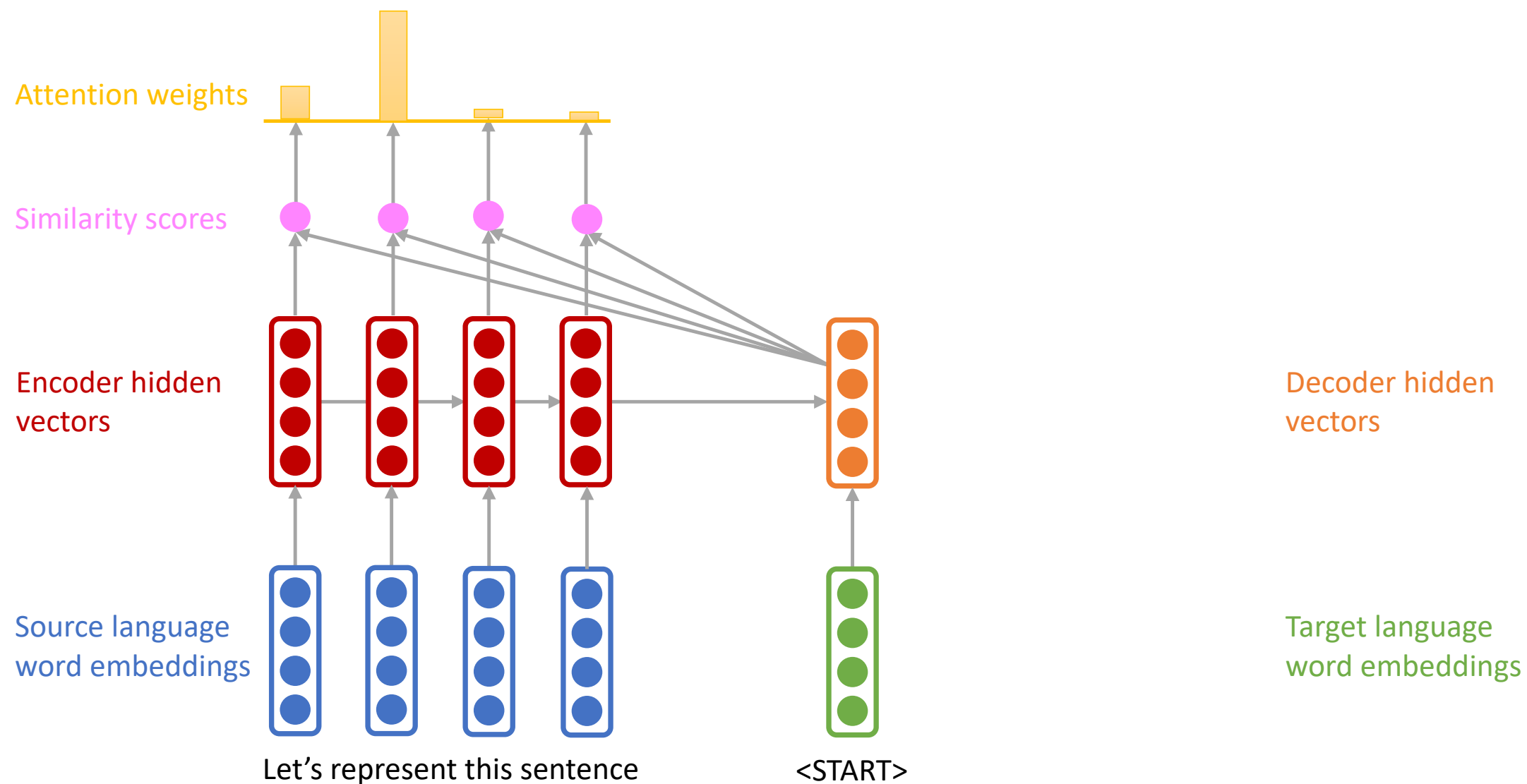
Calculate a **similarity score** between the decoder hidden vector and all the encoder hidden vectors.





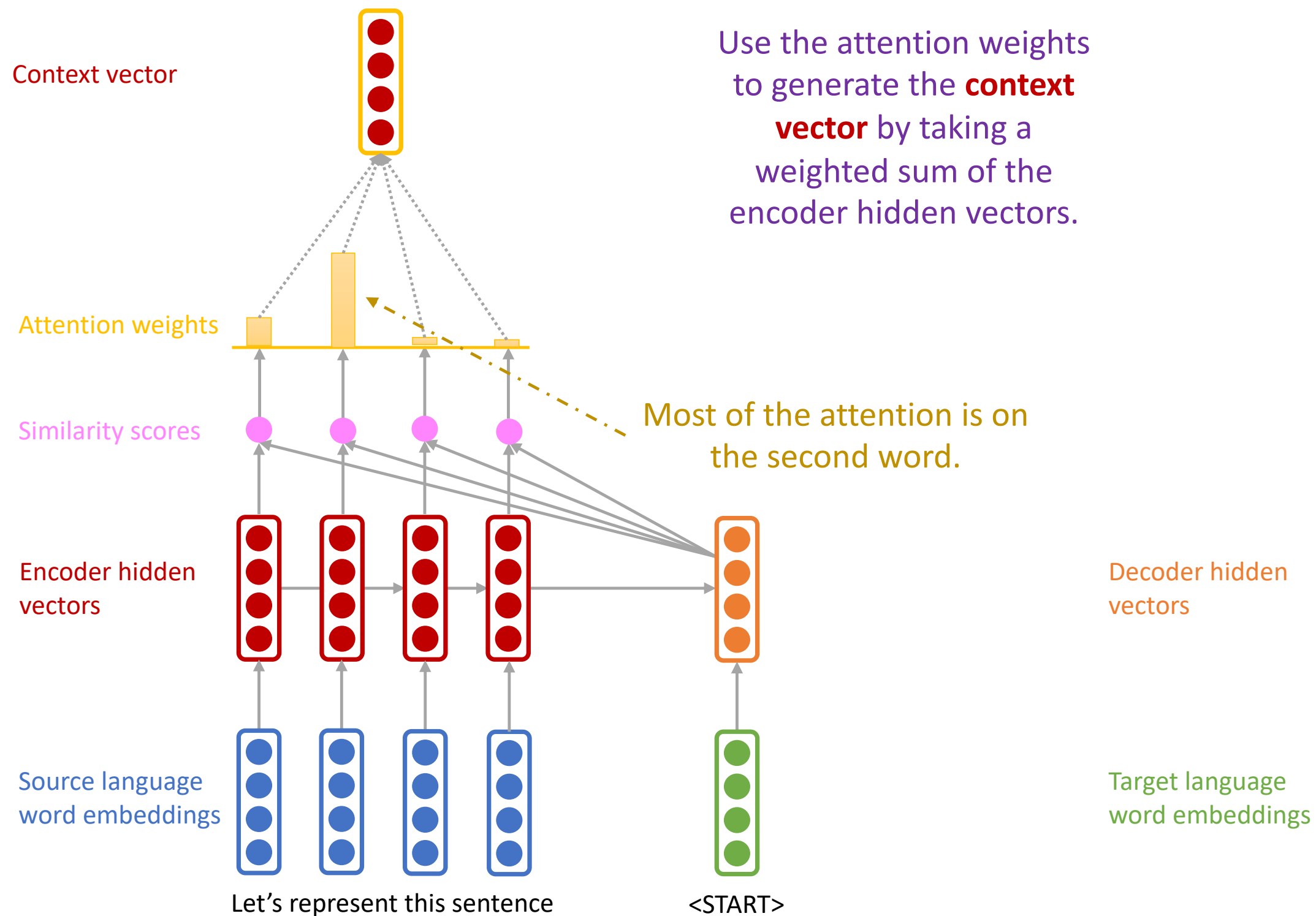
Seq2Seq with Attention: softmax similarity scores to get attention weight

Pass similarity score vector through a softmax to generate **attention weights**.



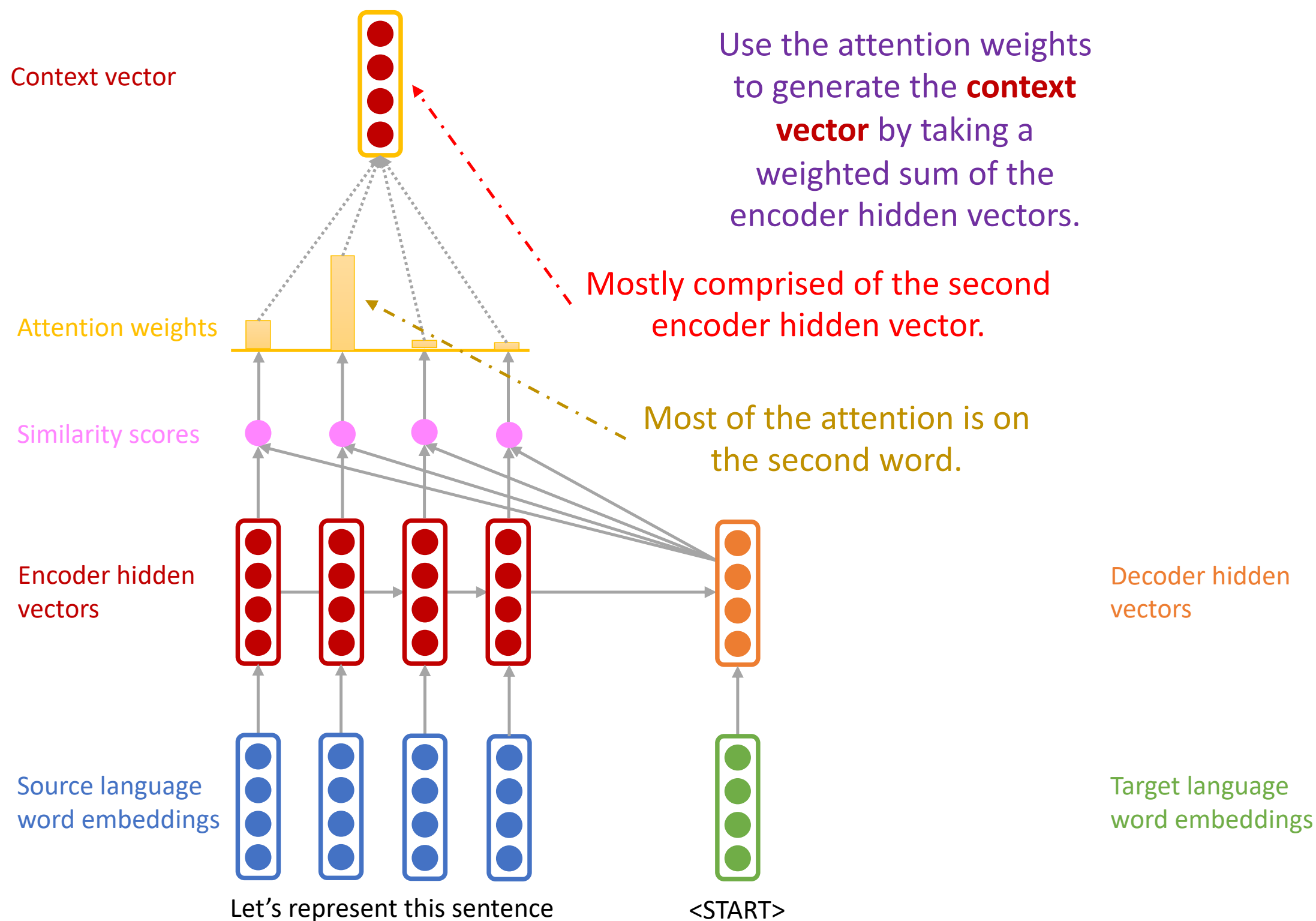


Softmax will provide a higher probability whenever there is an alignment





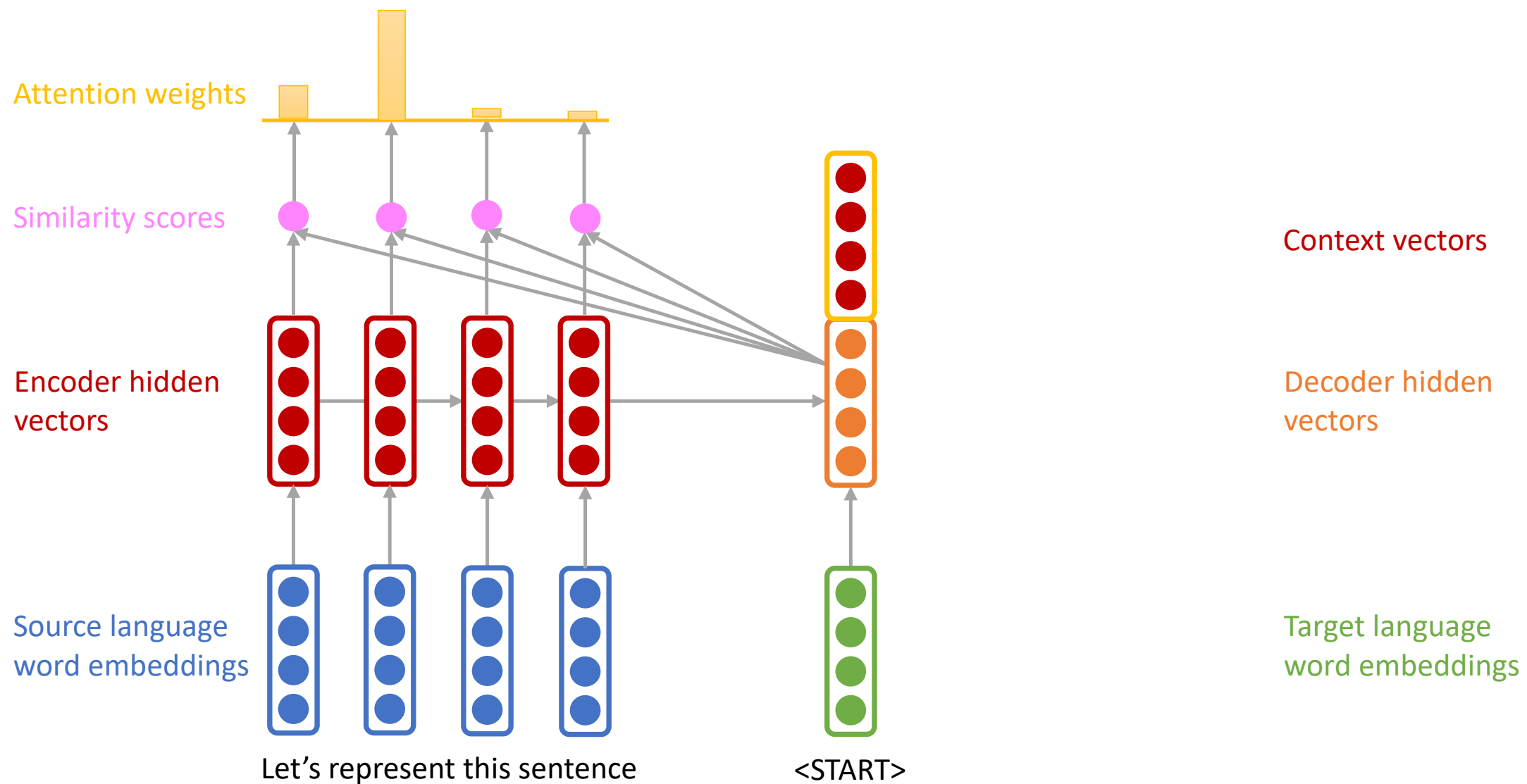
Selective summary





Add context vector to decoder hidden state

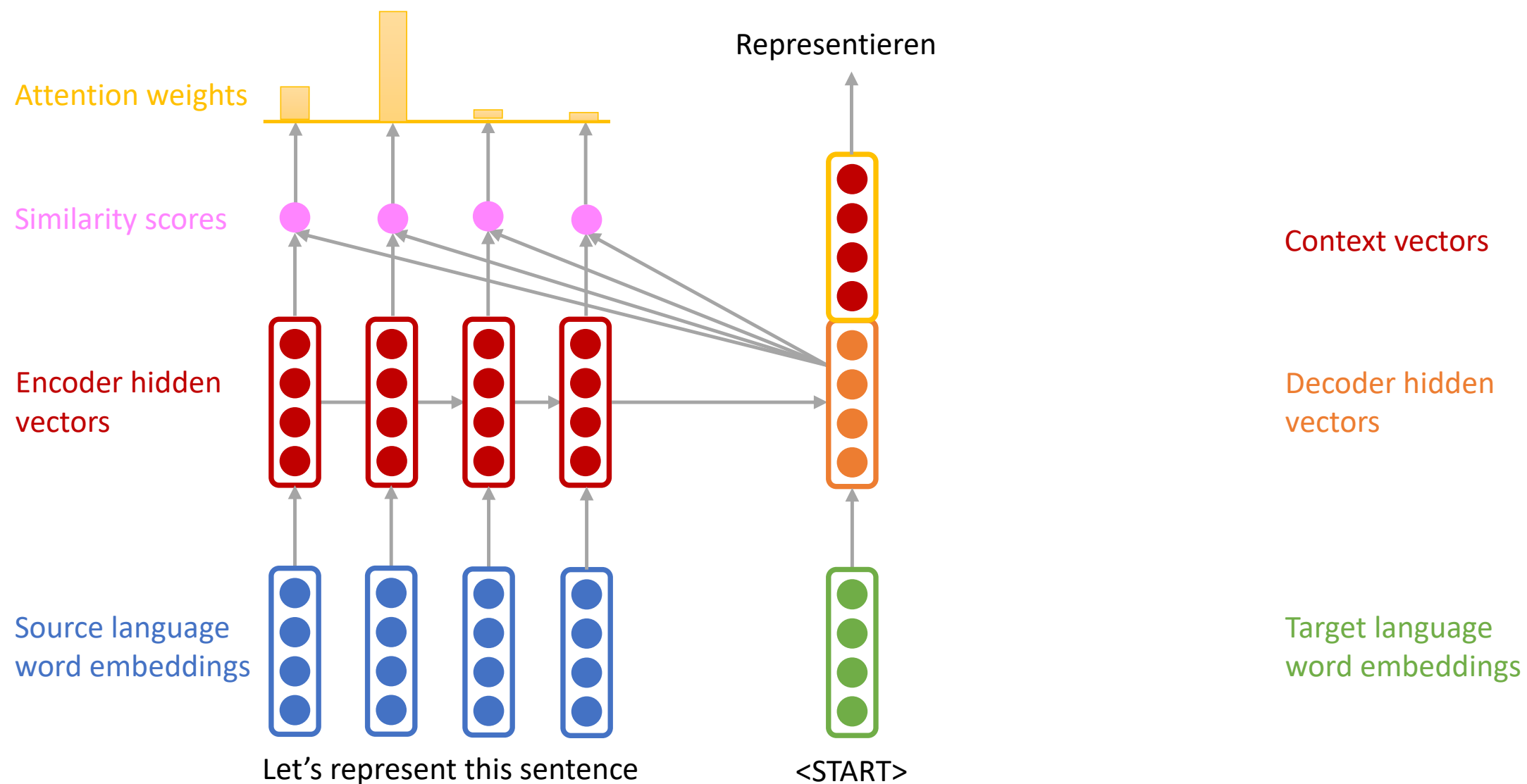
Concatenate the **context**
vector to the **decoder**
hidden vector





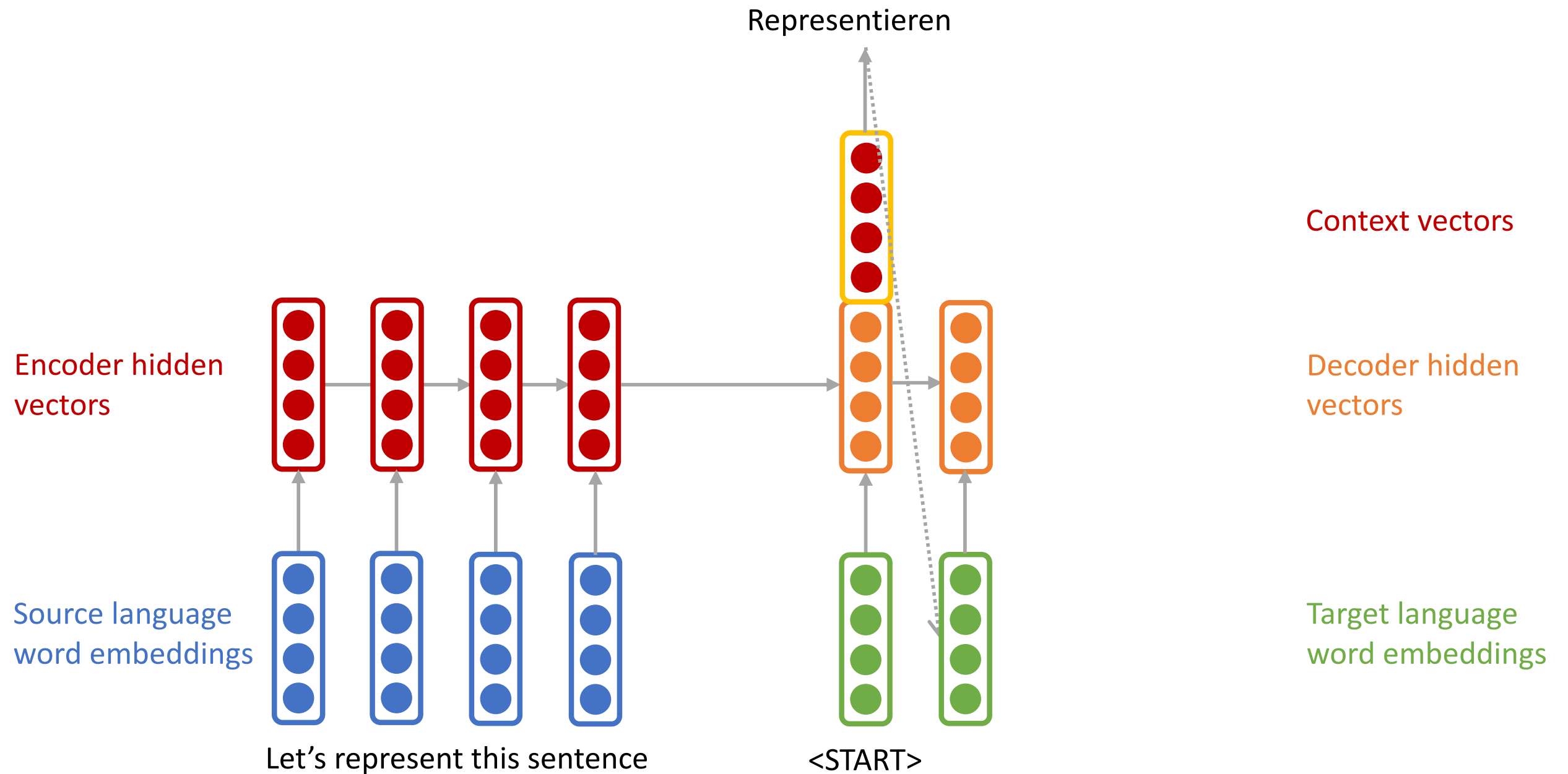
Generate next word

Concatenate the **context vector** to the **decoder hidden vector** and generate the first word.



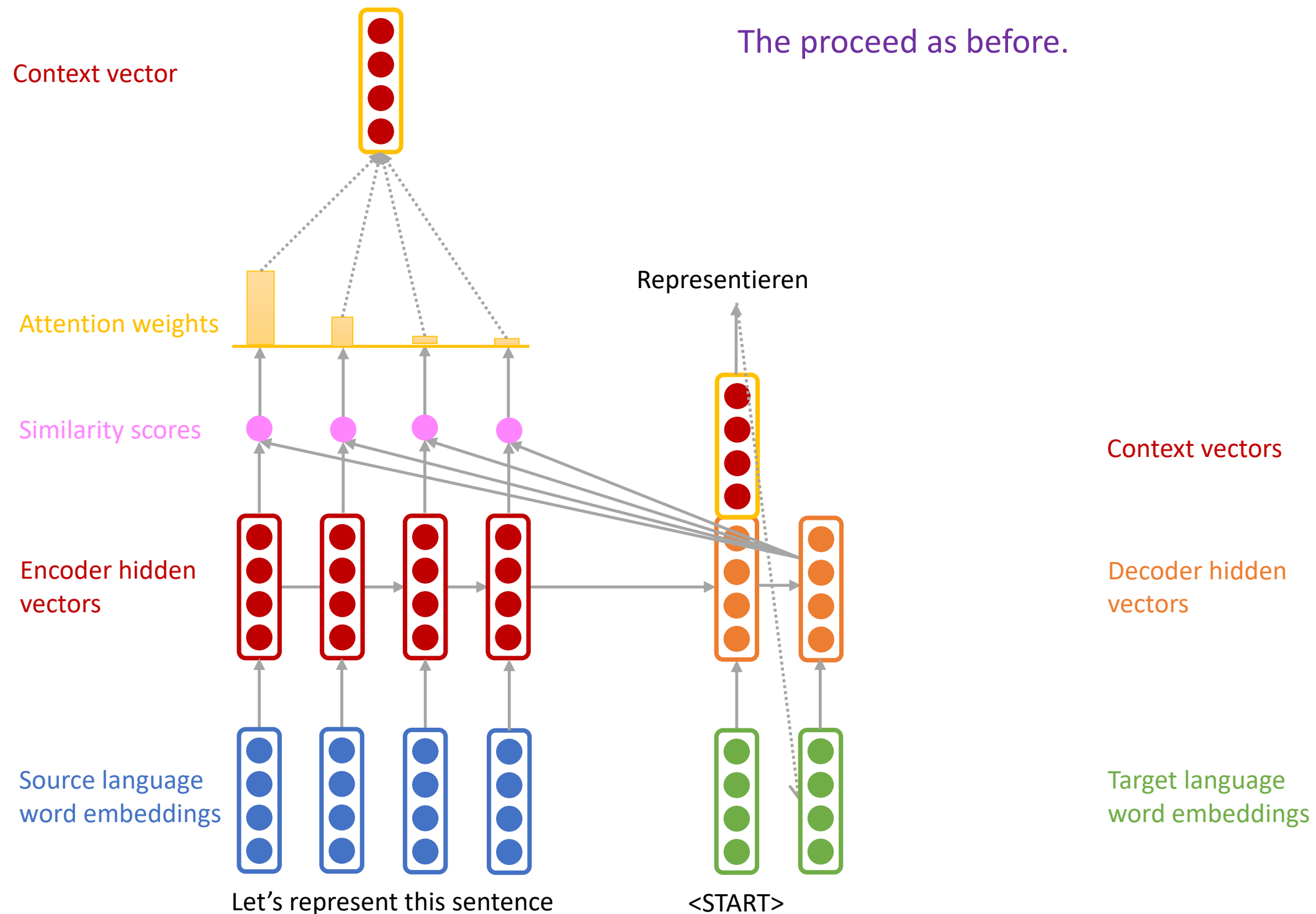


Proceed as before



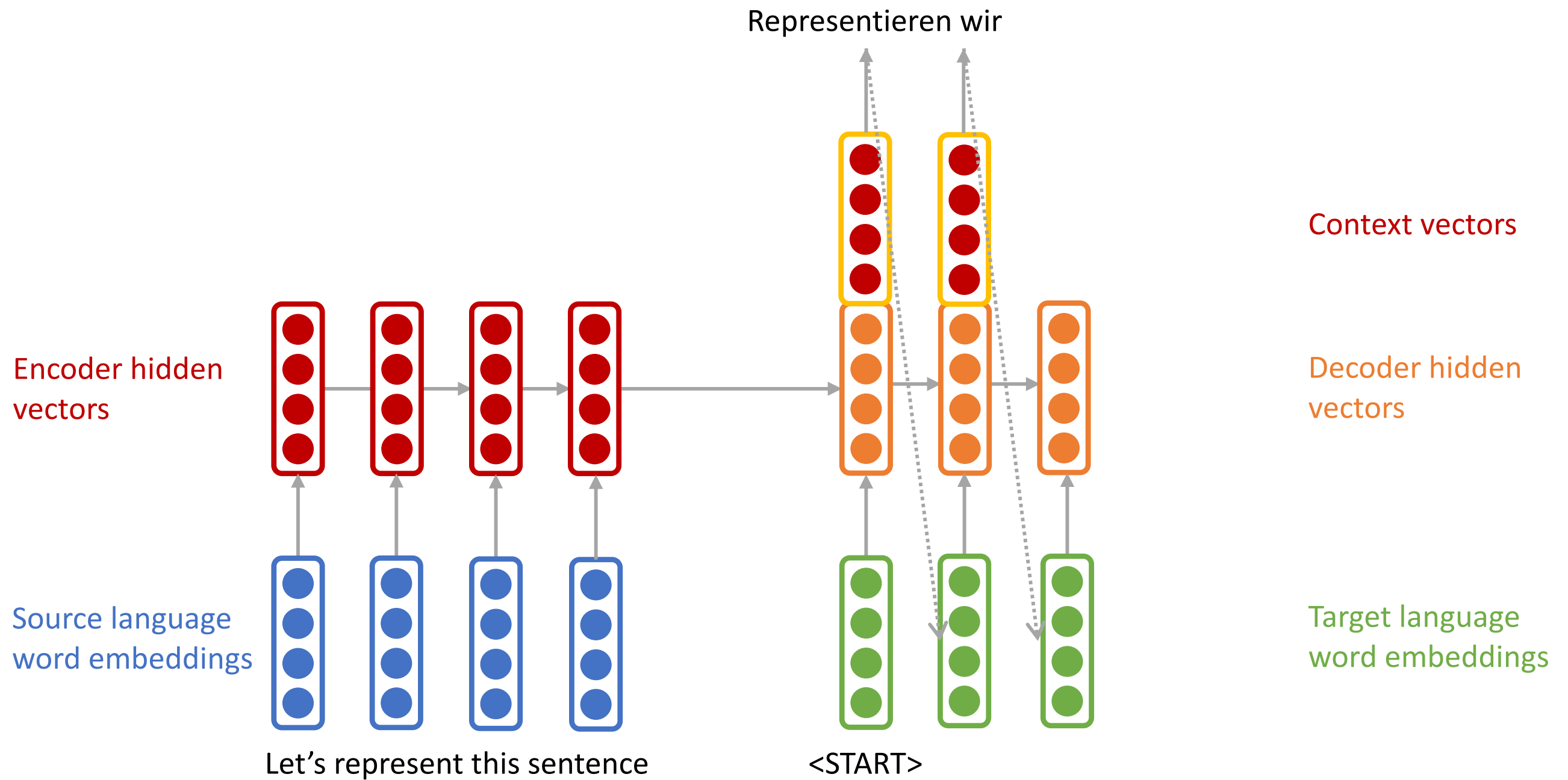


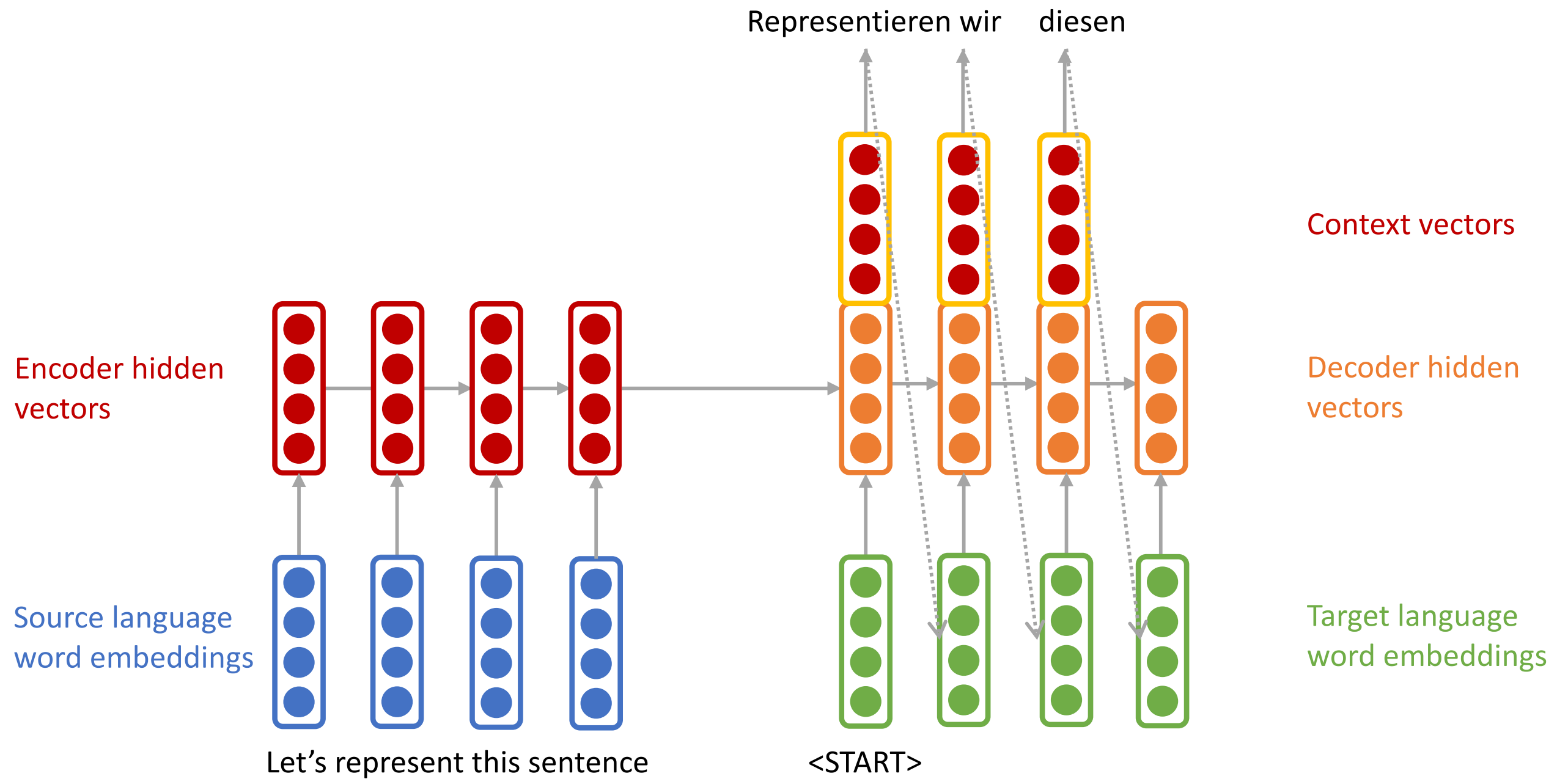
Selective context vector for each time step

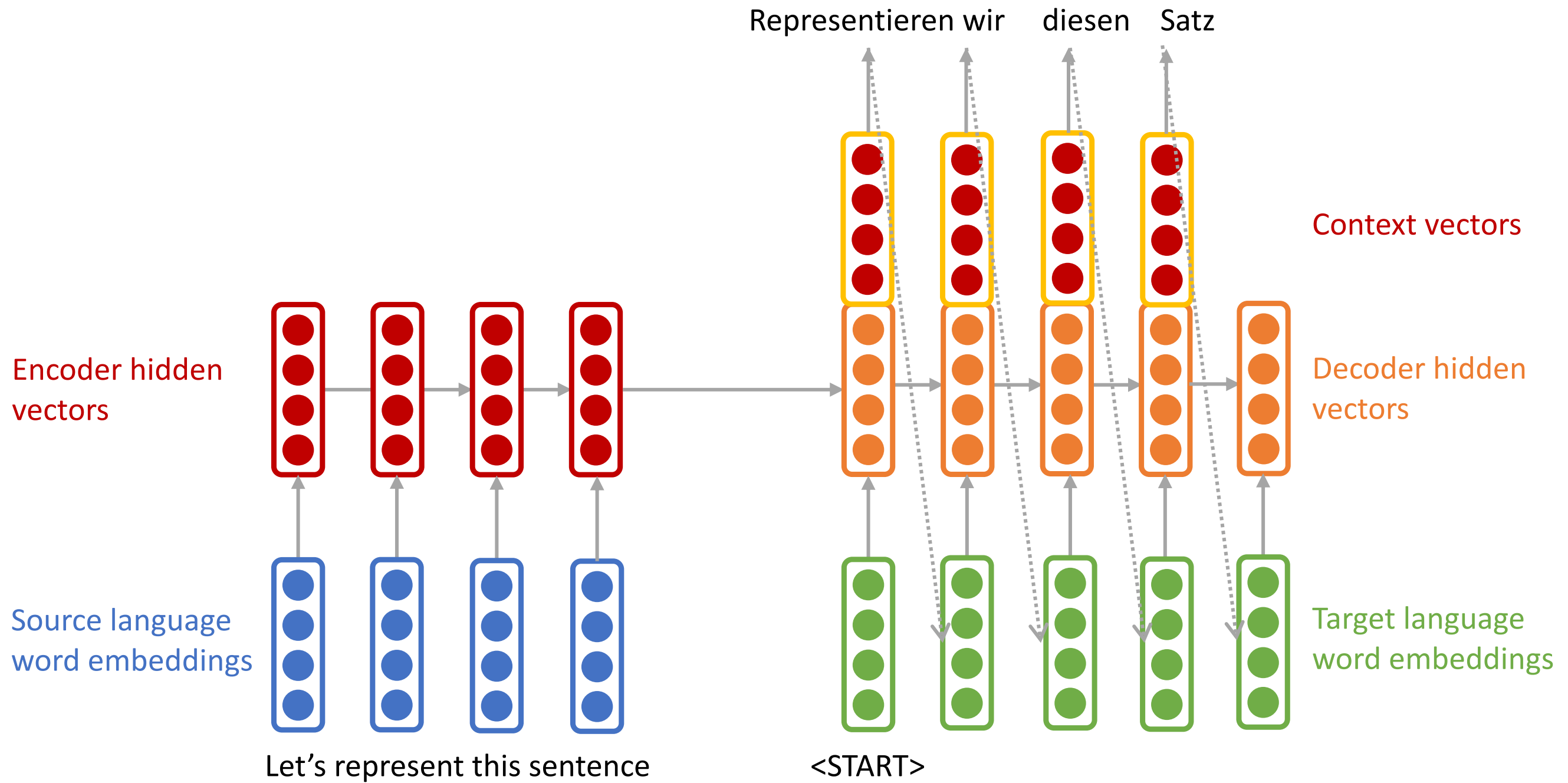




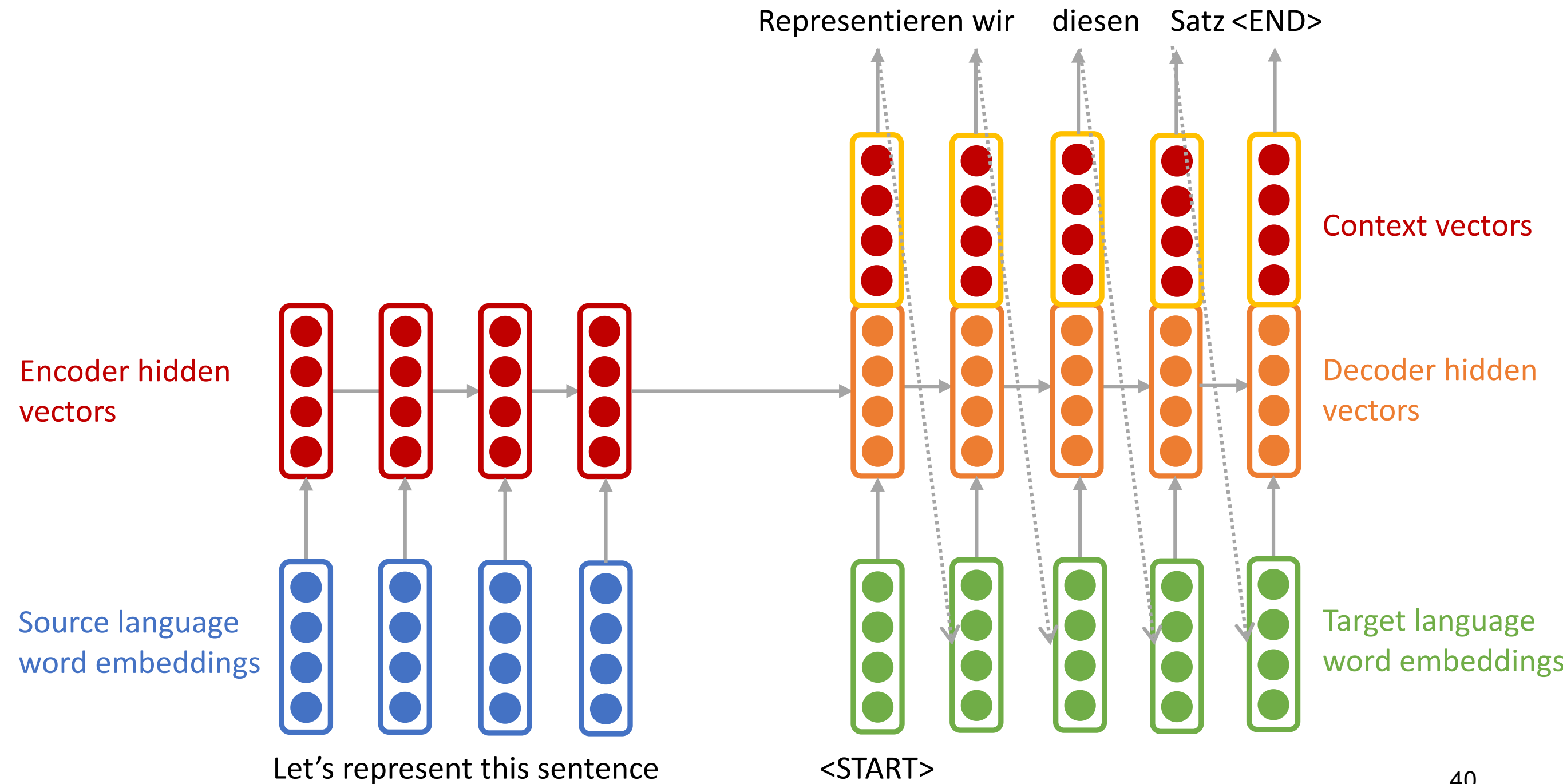
Generate next word







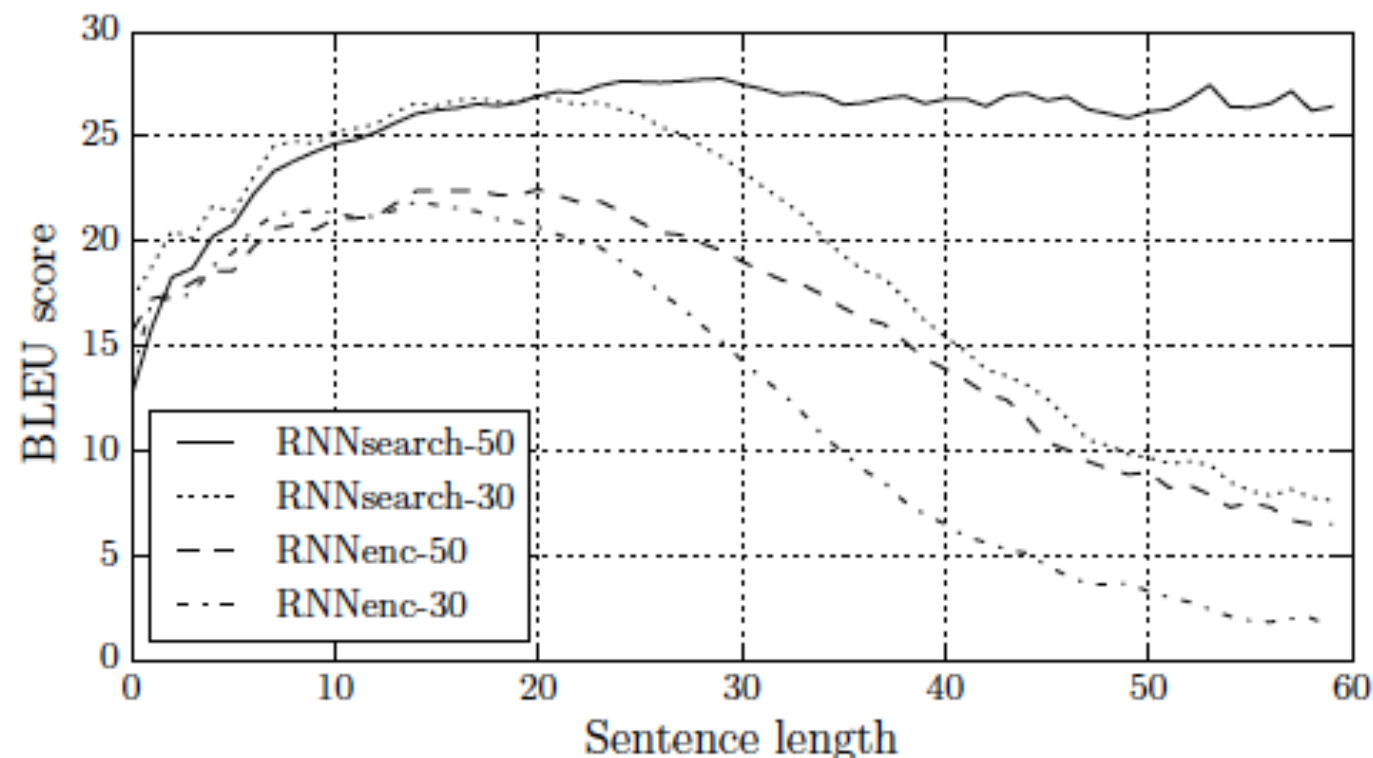
Translation is complete
when the <END> token is
generated.





Attention Machine Translation performance

Bahdanau, Cho, Bengio (ICLR-2015)



RNNsearch: with attention
RNNenc: no attention

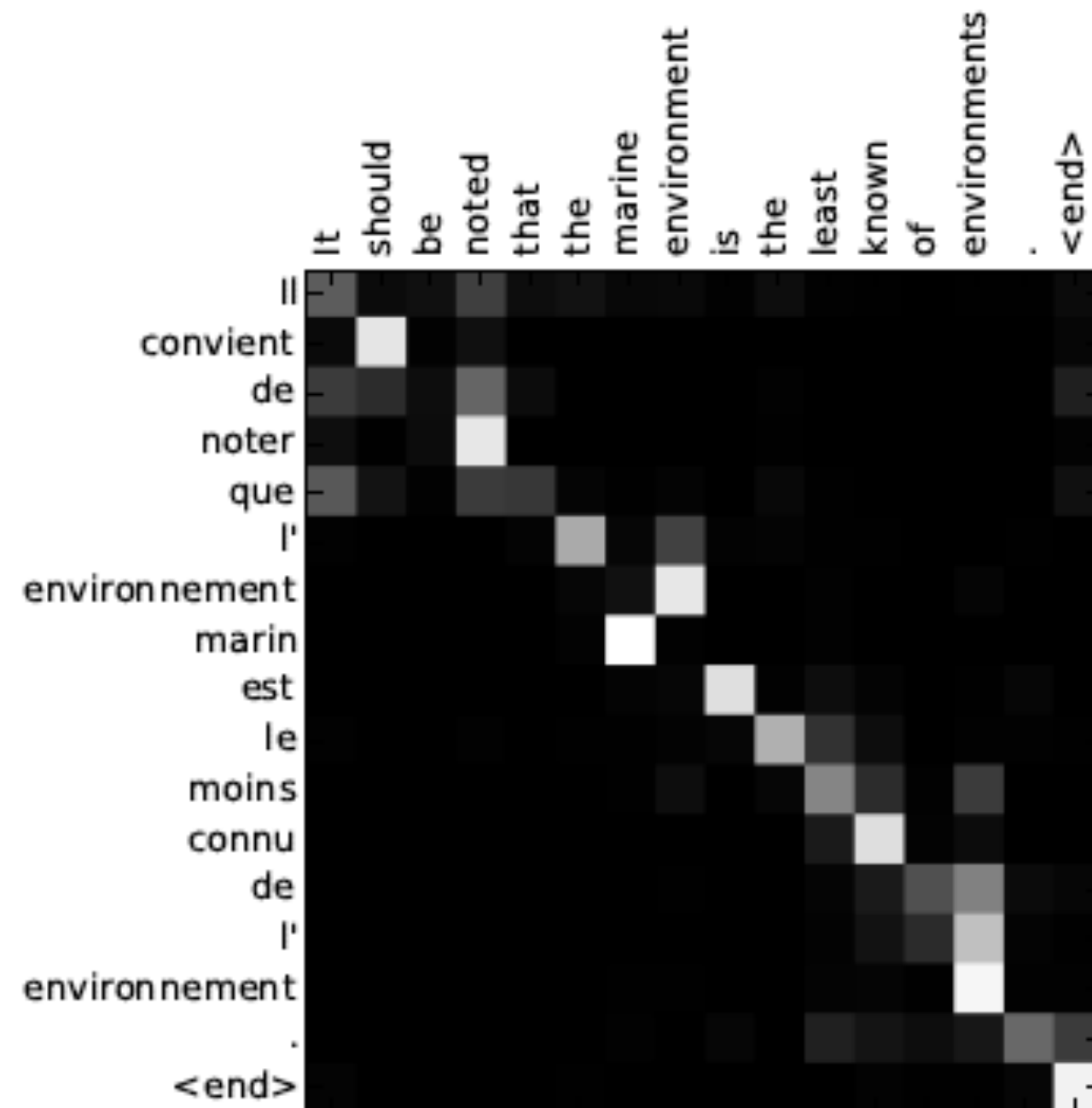
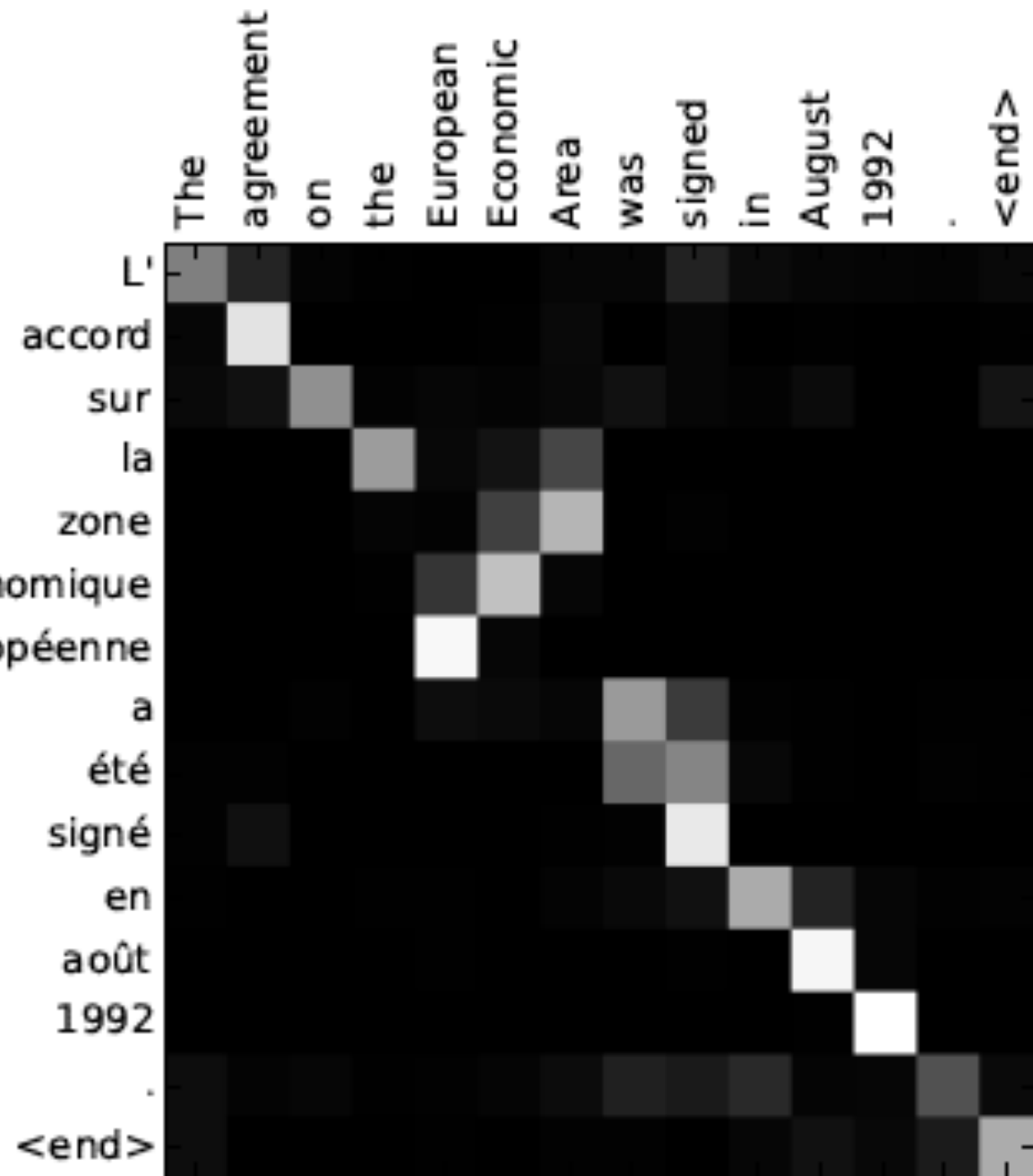
Bleu: BiLingual Evaluation Understudy

- Percentage of translated words that appear in ground truth



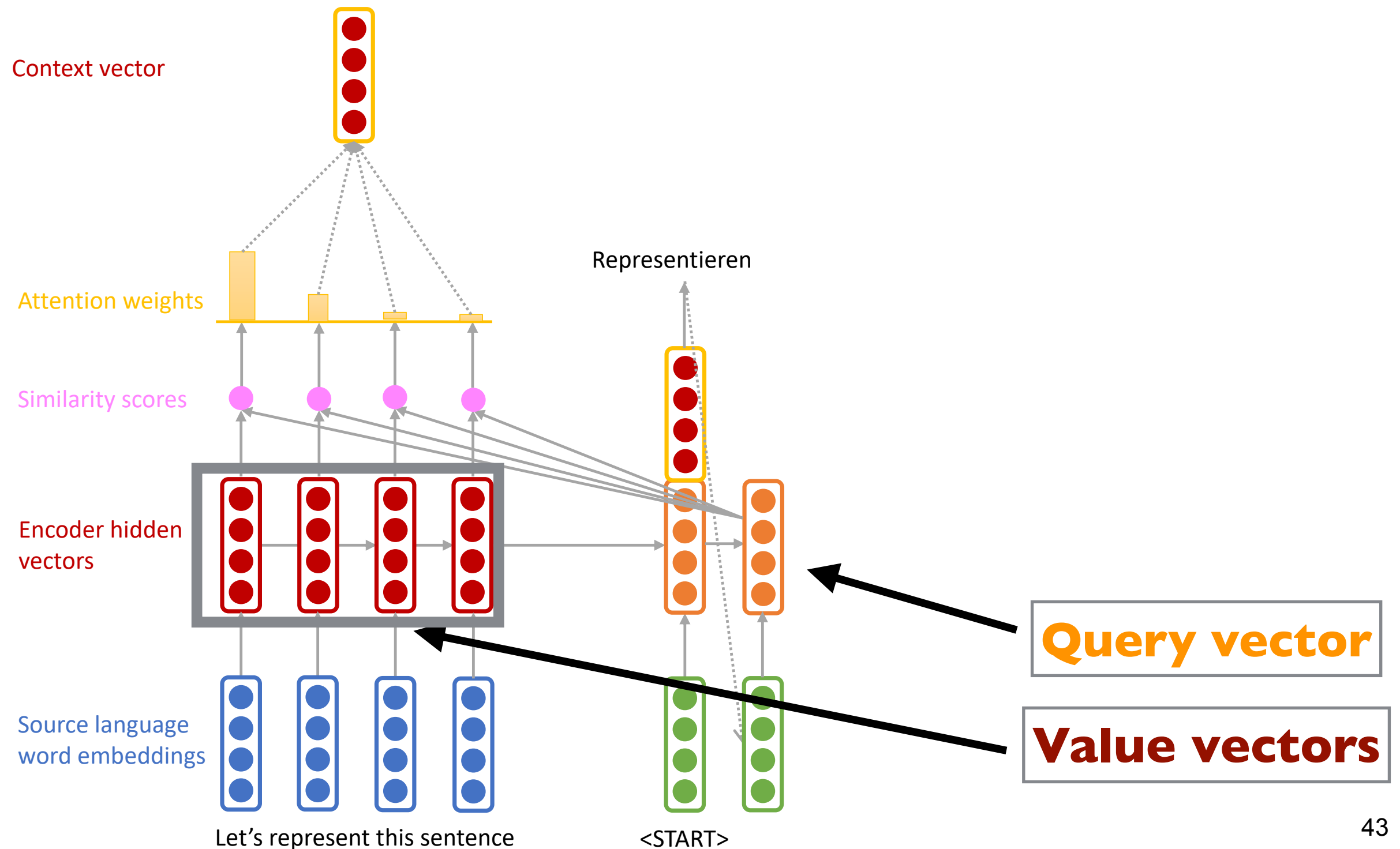
Alignment Example

Bahdanau, Cho, Bengio (ICLR-2015)



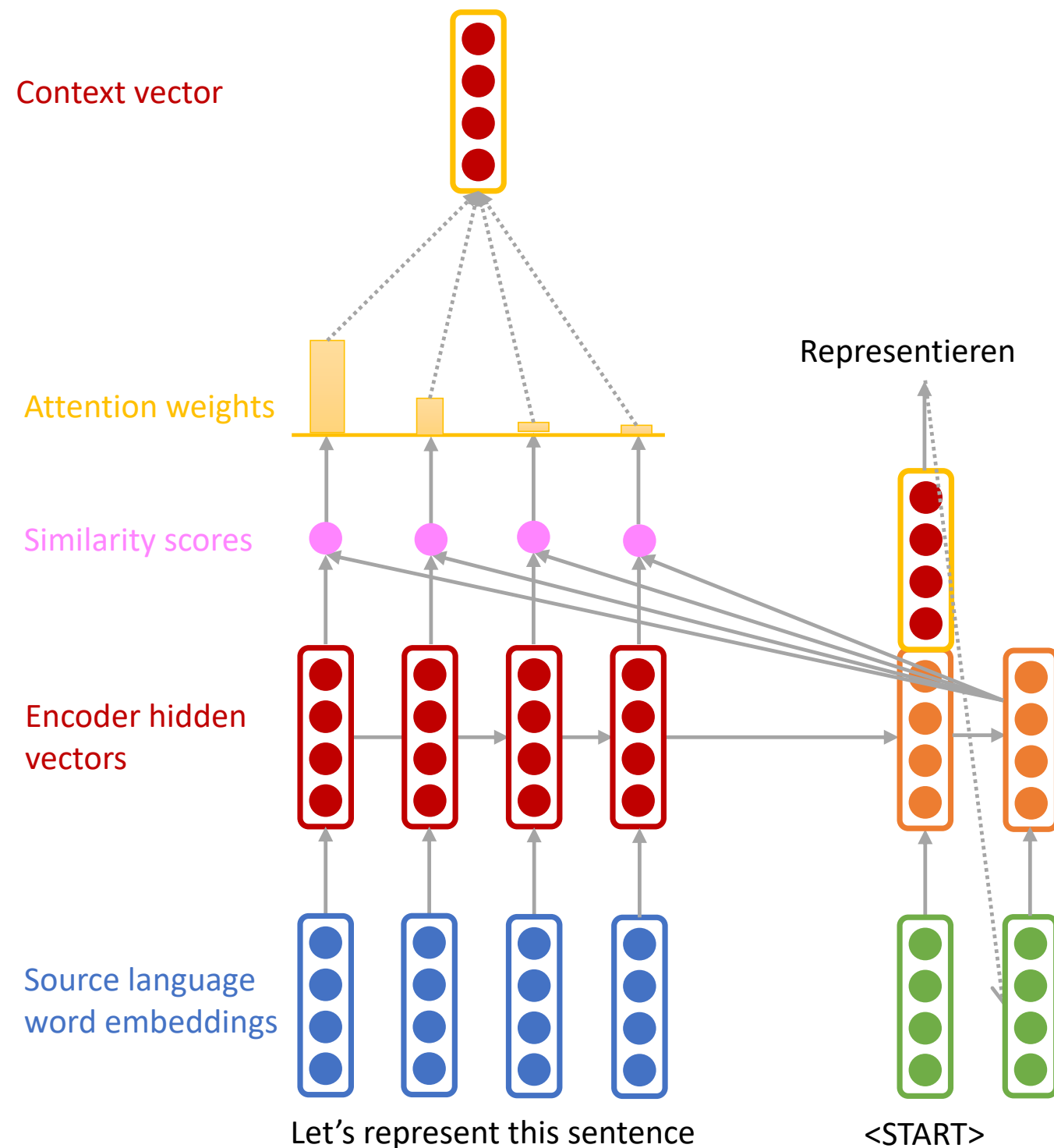


Important terminology: query vector, and value vectors





Important terminology: query vector, and value vectors





Attention is a general Deep Learning technique

We have some *values* $\mathbf{h}_1, \dots, \mathbf{h}_N \in \mathbb{R}^{d_1}$ and a *query* $\mathbf{s} \in \mathbb{R}^{d_2}$

Attention always involves:

1. Computing the *attention scores* $\mathbf{e} \in \mathbb{R}^N$
2. Taking softmax to get *attention distribution* α :

There are
multiple ways
to do this

$$\alpha = \text{softmax}(\mathbf{e}) \in \mathbb{R}^N$$

3. Using attention distribution to take weighted sum of values:

$$\mathbf{a} = \sum_{i=1}^N \alpha_i \mathbf{h}_i \in \mathbb{R}^{d_1}$$

thus obtaining the *attention output* $\mathbf{a}$ (sometimes called the *context vector*)



Attention: the many forms

There are **several ways** you can compute $e \in \mathbb{R}^N$ from $\mathbf{h}_1, \dots, \mathbf{h}_N \in \mathbb{R}^{d_1}$ and $\mathbf{s} \in \mathbb{R}^{d_2}$:

Basic dot-product attention: $e_i = \mathbf{s}^T \mathbf{h}_i \in \mathbb{R}$

- Note: this assumes $d_1 = d_2$.

Multiplicative attention: $e_i = \mathbf{s}^T \mathbf{W} \mathbf{h}_i \in \mathbb{R}$ [Luong, Pham, and Manning 2015]

- Where $\mathbf{W} \in \mathbb{R}^{d_2 \times d_1}$ is a weight matrix. Perhaps better called “bilinear attention”

Reduced-rank multiplicative attention: $e_i = \mathbf{s}^T (\mathbf{U}^T \mathbf{V}) \mathbf{h}_i = (\mathbf{U} \mathbf{s})^T (\mathbf{V} \mathbf{h}_i)$

- For low rank matrices $\mathbf{U} \in \mathbb{R}^{k \times d_2}$, $\mathbf{V} \in \mathbb{R}^{k \times d_1}$, $k \ll d_1, d_2$

Additive attention: $e_i = \mathbf{v}^T \tanh(\mathbf{W}_1 \mathbf{h}_i + \mathbf{W}_2 \mathbf{s}) \in \mathbb{R}$ [Bahdanau, Cho, and Bengio 2014]

- Where $\mathbf{W}_1 \in \mathbb{R}^{d_3 \times d_1}$, $\mathbf{W}_2 \in \mathbb{R}^{d_3 \times d_2}$ are weight matrices and $\mathbf{v} \in \mathbb{R}^{d_3}$ is a weight vector.
- d_3 (the attention dimensionality) is a hyperparameter
- “Additive” is a weird/bad name. It’s really using a feed-forward neural net layer.



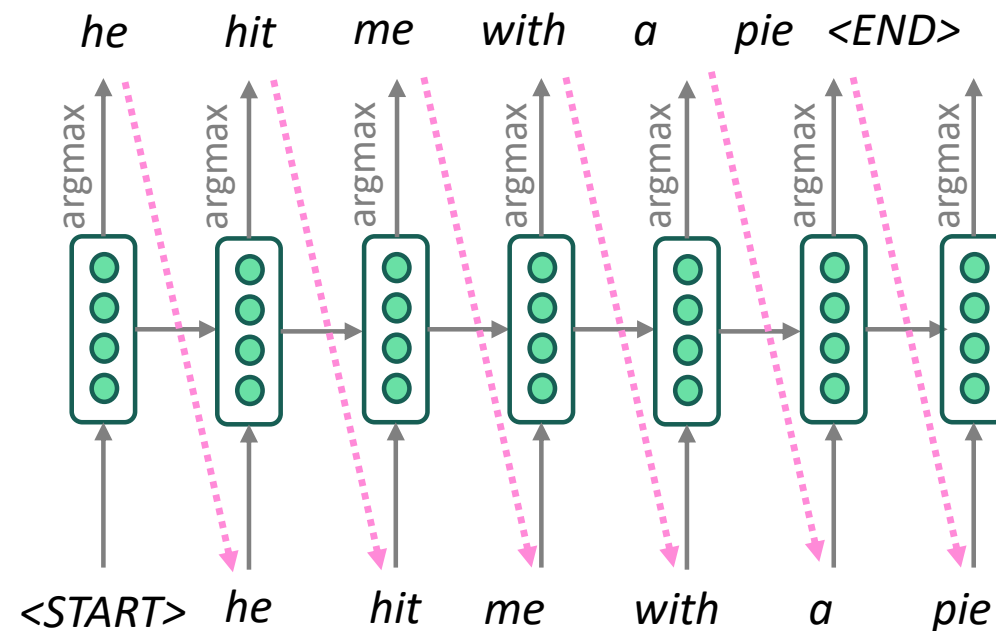
Decoding

- Greedy decoding is not great



Greedy decoding

- We saw how to generate (or “decode”) the target sentence by taking argmax on each step of the decoder



- This is **greedy decoding** (take most probable word on each step)
- **Problems with this method?**



Problems with greedy decoding

- Greedy decoding has no way to undo decisions!
 - Input: *il a m'entarté* (he hit me with a pie)
 - → *he* _____
 - → *he hit* _____
 - → *he hit* ***a*** _____ *(whoops! no going back now...)*
- How to fix this?



Exhaustive search decoding

- Ideally, we want to find a (length T) translation y that maximizes

$$\begin{aligned} P(y|x) &= P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots, P(y_T|y_1, \dots, y_{T-1}, x) \\ &= \prod_{t=1}^T P(y_t|y_1, \dots, y_{t-1}, x) \end{aligned}$$

- We could try computing **all possible sequences** y
 - This means that on each step t of the decoder, we're tracking V^t possible partial translations, where V is vocab size
 - This $O(V^T)$ complexity is **far too expensive!**

e.g, vocab size, 50k, and $t=3$,

There are: 125,000,000,000,000 possible sequences!!!



Beam search decoding

Core idea: On each step of decoder, keep track of the k most probable partial translations (which we call *hypotheses*)

- k is the *beam size* (in practice around 5 to 10)

A hypothesis $y_1, \dots, y_t$ has a *score* which is its log probability:

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- Scores are all negative, and higher score is better
- We search for high-scoring hypotheses, tracking top k on each step

Beam search is *not guaranteed* to find optimal solution

But *much more efficient* than exhaustive search!



Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

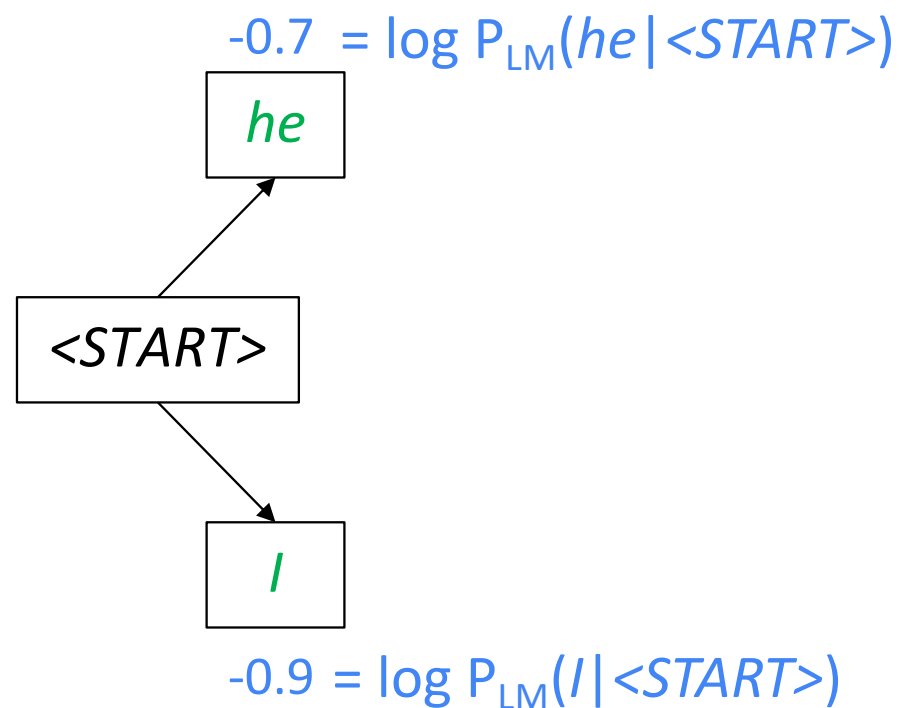
<START>

Calculate prob
dist of next word



Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$

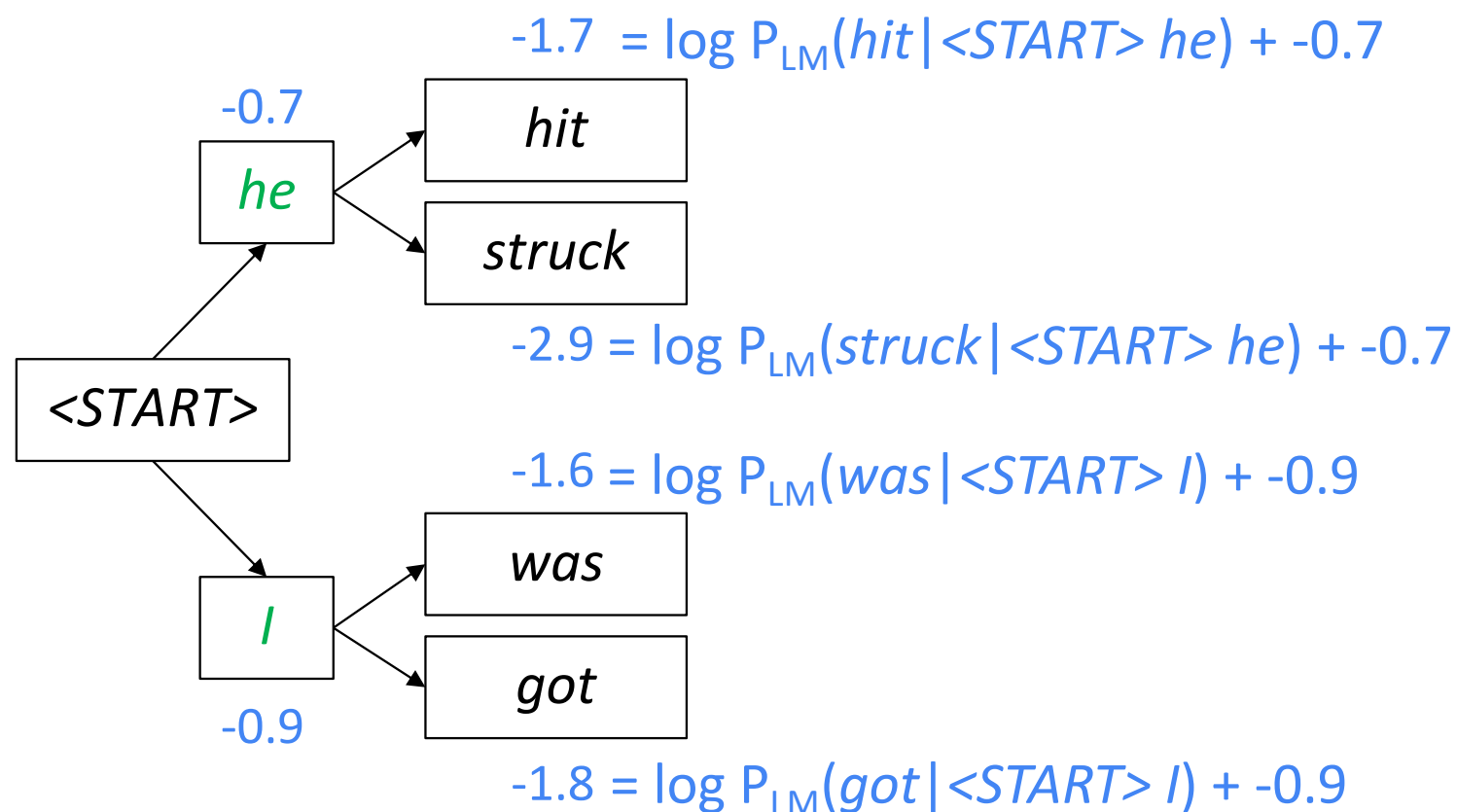


Take top k words
and compute scores



Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



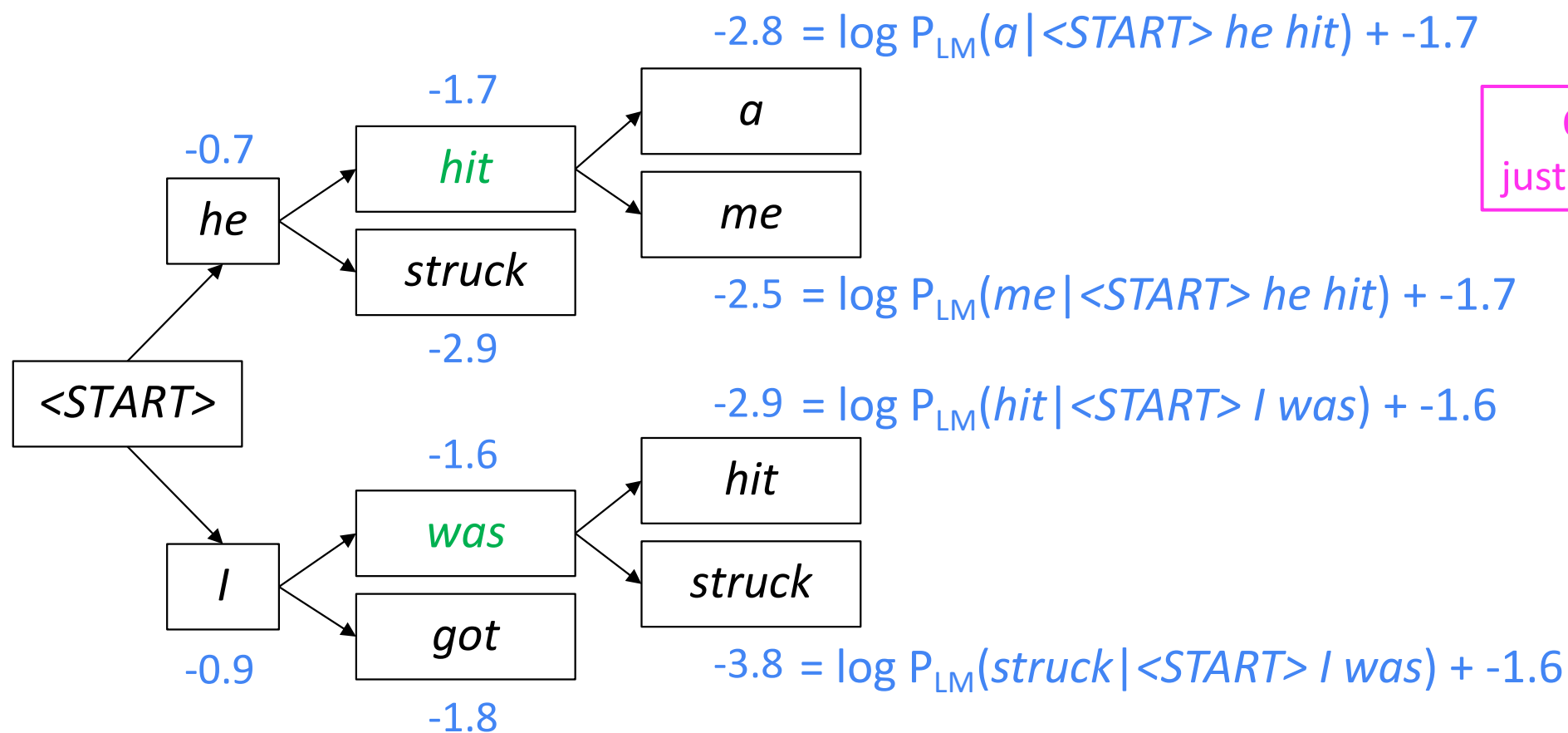
Of these k^2 hypotheses,
just keep k with highest scores

For each of the k hypotheses, find
top k next words and calculate scores



Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



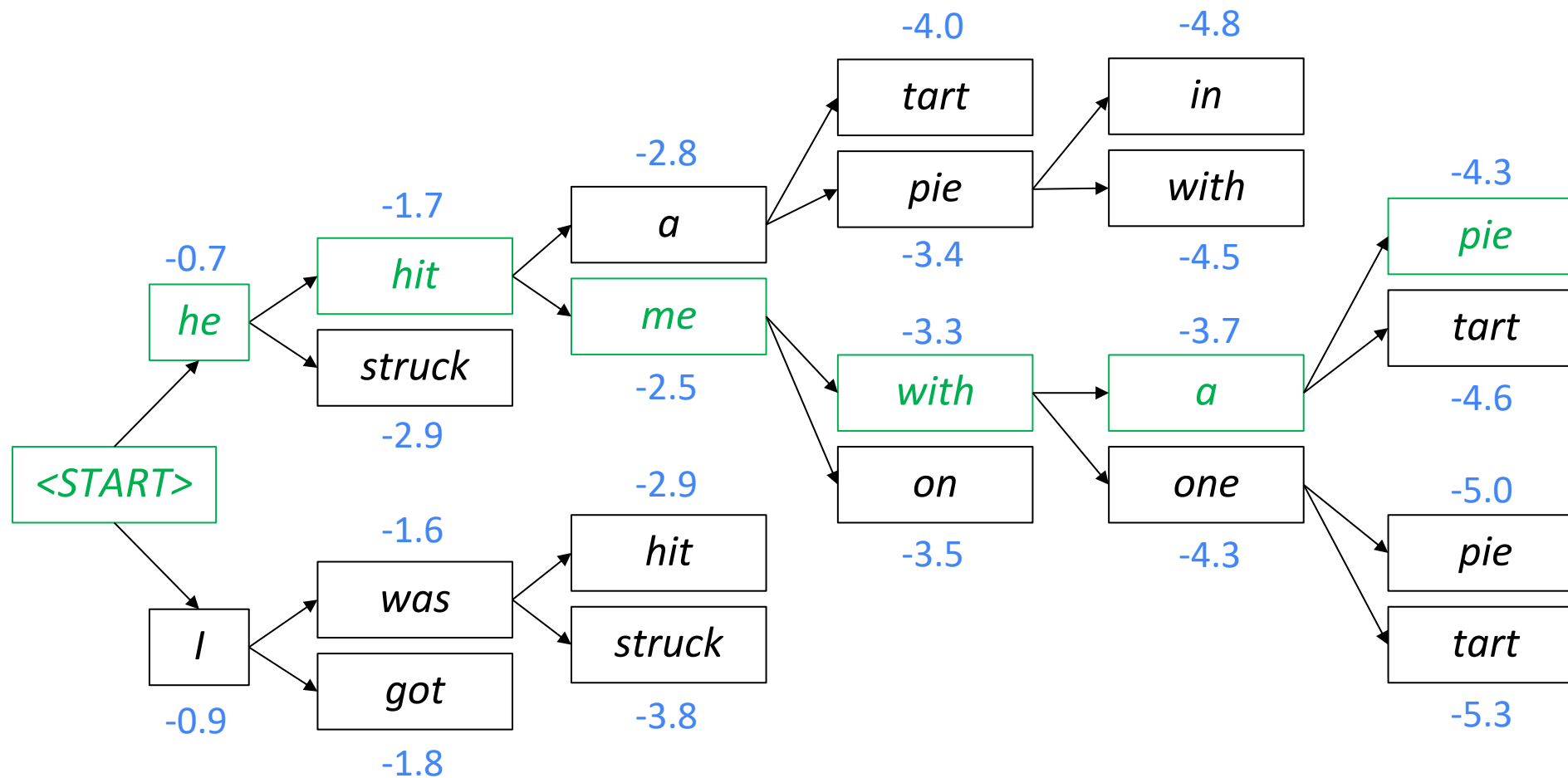
Of these k^2 hypotheses, just keep k with highest scores

For each of the k hypotheses, find top k next words and calculate scores



Beam search decoding: example

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Backtrack to obtain the full hypothesis



Beam search decoding: stopping criterion

In **greedy decoding**, usually we decode until the model produces an **<END> token**

- **For example:** *<START> he hit me with a pie <END>*

In **beam search decoding**, different hypotheses may produce **<END> tokens** on **different timesteps**

- When a hypothesis produces **<END>**, that hypothesis is **complete**.
- **Place it aside** and continue exploring other hypotheses via beam search.

Usually we continue beam search until:

- We reach timestep T (where T is some pre-defined cutoff), or
- We have at least n completed hypotheses (where n is pre-defined cutoff)



Beam search decoding: finishing up

We have our list of completed hypotheses.

How to select top one with highest score?

Each hypothesis $y_1, \dots, y_t$ on our list has a score

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

Problem with this: longer hypotheses have lower scores

Fix: Normalize by length. Use this to select top one instead:

$$\frac{1}{t} \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$



Disadvantages of NMT?

Compared to SMT:

- NMT is **less interpretable**
 - Hard to debug
- NMT is **difficult to control**
 - For example, can't easily specify rules or guidelines for translation



Summary

- Since 2014, **Neural MT** rapidly replaced intricate Statistical MT
- **Sequence-to-sequence** is the architecture for NMT (uses 2 models: encoder and decoder)
- **Attention** is a way to *focus on particular parts* of the input
 - Improves sequence-to-sequence a lot!





NMT: the first big success story of NLP Deep Learning?

Neural Machine Translation went from a **fringe research attempt** in **2014** to the **leading standard method** in **2016**

- **2014**: First seq2seq paper published
- **2016**: Google Translate switches from SMT to NMT – and by 2018 everyone has



- **This is amazing!**
 - **SMT** systems, built by **hundreds** of engineers over many **years**, outperformed by NMT systems trained by a **small group** of engineers in a few **months**



Questions ?

- Includes slides from
 - Abigail See
 - Many other directly or indirectly



Bonus slides

- From here ..



Frontiers in MT: Is ChatGPT a good translator?

Table 3: Comparison of different prompts for ChatGPT to perform Chinese-to-English (Zh $\Rightarrow$ En) translation.

System	BLEU $\uparrow$	ChrF++ $\uparrow$	TER $\downarrow$
Google	31.66	57.09	56.21
DeepL	31.22	56.74	57.84
Tencent	29.69	56.24	57.16
ChatGPT w/ TP1	23.25	53.07	66.03
ChatGPT w/ TP2	24.54	53.05	63.79
ChatGPT w/ TP3	24.73	53.71	62.84

- Works okay for Chinese-English, but less good at generating into low-resource languages (English $\rightarrow$ Romanian doesn't work well)

"Is ChatGPT A Good Translator? Yes With GPT-4 As The Engine" Jia et al. (2023)



MT for low-resource languages

Burmese, Indonesian, Turkish BLEU

Transfer	My→En	Id→En	Tr→En
baseline (no transfer)	4.0	20.6	19.0
transfer, train	17.8	27.4	20.3
transfer, train, reset emb, train	13.3	25.0	20.0
transfer, train, reset inner, train	3.6	18.0	19.1

Aji et al. (2020)

Table 3: Investigating the model's capability to restore its quality if we reset the parameters. We use En→De as the parent.

- Interest in deploying MT systems for languages with little or no data
- BPE allows us to transfer models even without training on a specific language



How do we evaluate MT

- We can ask a human to do evaluation, but slow, expensive, and sometimes inconsistent
- Or use automated metrics like BLEU



Evaluation of Machine Translation Systems

- What makes a good translation?
- Intuition: good translations are **fluent** in the target language and **faithful** to the original meaning

“A Vinay le gusta Python”

	Adequate?	Fluent?
<i>To Vinay it like Python</i>	yes	no
<i>Vinay debugs memory leaks</i>	no	yes
<i>Vinay likes Python</i>	yes	yes



MT Evaluation: BLUE

- **BLEU** - bilingual evaluation understudy (Papineni et., al 2002)
 - ▶ Compare to a human-generated reference translation
 - ▶ Or, better: multiple references
 - ▶ Weighted average of n-gram precision (across different n)

“A Vinay le gusta Python”

	Adequate?	Fluent?
<i>To Vinay it like Python</i>	yes	no
<i>Vinay debugs memory leaks</i>	no	yes
<i>Vinay likes Python</i>	yes	yes



BLEU limitations

- **BLEU** - bilingual evaluation understudy (Papineni et., al 2002)
- BLEU is **useful** but **imperfect**
 - Easy to use, good for measuring system improvement
 - But, there are many valid ways to translate a sentence
 - A good translation can get a poor BLEU score because of low n-gram overlap with the human translation



Embedding-based Metrics

- Recent: metrics based on neural models
 - **BertScore:** Find similarity between BERT embeddings (unsupervised) (Zhang et al. 2020)
 - **BLEURT:** Train BERT to predict human evaluation scores (Sellam et al.
 - ...



Frontiers: Evaluation with LLMs

Score the following translation from {source_lang} to {target_lang} **with respect to the human reference** on a continuous scale from 0 to 100, where score of zero means "no meaning preserved" and score of one hundred means "perfect meaning and grammar"

```
{source_lang} source: "{source_seg}"  
{target_lang} human reference: {reference_seg}  
{target_lang} translation: "{target_seg}"  
Score:
```

Figure 1: The best-performing prompt based on Direct Assessment expecting a score between 0–100. Template **portions in bold face** are used only when a human reference translation is available.

- Outperforms many learned MT metrics (Transformers trained over (source, target, reference) triples to reproduce human judgments of quality)

“Large Language Models Are State-of-the-Art Evaluators of Translation Quality” Kocmi et al. (2023)